

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

MATHEUS KUNNEN LEDESMA

**FRAMEWORK PON MICROPYTHON 1.0: PARADIGMA ORIENTADO A
NOTIFICAÇÕES PARA MICROCONTROLADORES**

**CURITIBA
2025**

MATHEUS KUNNEN LEDESMA

**FRAMEWORK PON MICROPYTHON 1.0: PARADIGMA ORIENTADO A
NOTIFICAÇÕES PARA MICROCONTROLADORES**

**FRAMEWORK NOP MICROPYTHON 1.0: NOTIFICATION ORIENTED PARADIGM
FOR MICROCONTROLLERS**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do
título de Bacharel em Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Orientador(a): Dr. Jean Marcelo Simão.
Coorientador(a): Dr. João Alberto Fabro.

CURITIBA

2025



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

MATHEUS KUNNEN LEDESMA

**FRAMEWORK PON MICROPYTHON 1.0: PARADIGMA ORIENTADO A
NOTIFICAÇÕES PARA MICROCONTROLADORES**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do
título de Bacharel em Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Data de aprovação: 17/fevereiro/2025

Ana Cristina Barreiras Kochem Vendramin
Doutora
Universidade Tecnológica Federal do Paraná

Douglas Paulo Bertrand Renaux
Doutor
Universidade Tecnológica Federal do Paraná

Jean Marcelo Simão
Doutor
Universidade Tecnológica Federal do Paraná

Paulo César Stadzisz
Doutor
Universidade Tecnológica Federal do Paraná

**CURITIBA
2025**

Dedico este trabalho aos meus pais, Natalia
Ledesma e Claudio Kunnen Doerner, pelo apoio
constante que nunca me faltou ao longo desta fase
da minha vida.
Este trabalho é tanto de vocês quanto meu.

AGRADECIMENTOS

A realização deste trabalho só foi possível graças ao apoio, inspiração e dedicação de muitas pessoas que me acompanharam durante essa jornada acadêmica.

Agradeço a minha família, especialmente a minha mãe e meu pai, por seu amor, paciência e apoio incondicional, sendo a base que sustentou todo o meu progresso.

Sou imensamente grato ao meu orientador Prof. Dr. Jean Marcelo Simão e ao meu coorientador Prof. Dr. João Alberto Fabro, pela orientação, incentivo e pelos ensinamentos compartilhados ao longo de todo o curso. Suas dedicações e conhecimentos foram fundamentais para a conclusão deste trabalho.

Aos professores e colegas do curso de Engenharia de Computação, por contribuírem com seus conhecimentos e ideias, além dos momentos de aprendizado e companheirismo que enriqueceram minha trajetória acadêmica e vida pessoal. Em especial, meu sincero agradecimento a Patricia Turato, Andrea Oquendo e Fernanda Rocha, por seu apoio, amizade e inspiração ao longo dessa jornada.

Aos meus amigos, pelo companheirismo, apoio e incentivo ao longo desta jornada. Em especial, a Andrea Larissa Ayala Torales, por seu carinho, incentivo e apoio incondicional durante toda essa caminhada. Sua presença foi essencial para que eu seguisse firme em cada etapa deste desafio.

Por fim, estendo meus agradecimentos a todas as pessoas que, de alguma forma, contribuíram para esta importante fase da minha vida, mas que não estão presentes entre essas palavras. Saibam que cada palavra de incentivo, conselho ou gesto de apoio foi essencial para alcançar este objetivo.

RESUMO

O Paradigma Orientado a Notificações (PON) surge como uma solução inovadora para superar limitações recorrentes em paradigmas de programação tradicionais, como redundância de código e alto acoplamento. A concretização dos conceitos do PON tem sido realizada por meio de *frameworks* no estado da técnica e pela Tecnologia LingPON no estado da arte. Este trabalho apresenta o desenvolvimento do *Framework* PON MicroPython 1.0, projetado especificamente para microcontroladores compatíveis com o interpretador MicroPython. Durante o processo de desenvolvimento, foram priorizadas a integração dos elementos do PON com os periféricos mais comuns em microcontroladores, como *General Purpose Input-Output* (GPIO), *Analog-to-Digital Converter* (ADC) e *Pulse Width Modulation* (PWM), além de um mecanismo modular para comunicação de notificações entre dispositivos distintos. Essa comunicação é viabilizada por meio de *drivers* de comunicação que preservam uma interface de programação de alto nível, garantindo usabilidade e flexibilidade. Os resultados obtidos incluem o desenvolvimento de duas aplicações práticas para validação do *framework*. A primeira aplicação tem como objetivo avaliar o custo da integração dos periféricos com os elementos do PON, tanto de maneira local quanto distribuída entre dois microcontroladores. Já a segunda aplicação, denominada *Air Quality Monitor*, explora o potencial do *framework* em simplificar a distribuição das notificações em um cenário mais complexo, envolvendo quatro microcontroladores e dois *drivers* de comunicação distintos. Ambas as aplicações demonstraram que o *framework* facilitou o desenvolvimento, reduzindo o tamanho do código final e mantendo uma interface de programação de alto nível. Contudo, os resultados também evidenciaram um aumento na latência entre as ações e no consumo de memória RAM, resultado das estruturas utilizadas pelo *framework* para fornecer as abstrações dos componentes do PON. Além disso, este trabalho detalha o uso dos elementos do *framework*, abrangendo tanto os integrados com periféricos quanto os próprios *drivers* de comunicação desenvolvidos. A documentação apresentada fornece uma base sólida para que outros desenvolvedores possam adotar e expandir o uso do *framework* em diferentes contextos. Por fim, são propostas melhorias futuras para o *Framework* PON MicroPython 1.0, com foco no aprimoramento do mecanismo de comunicação.

Palavras-chave: Programação Orientada a Notificações; Microcontroladores; *Framework* PON MicroPython; Sistemas Distribuídos.

ABSTRACT

The Notification-Oriented Paradigm (NOP) emerges as an innovative solution to overcome recurring limitations in traditional programming paradigms, such as code redundancy and high coupling. The realization of NOP concepts has been achieved through state-of-the-art frameworks and the state-of-the-art LingPON Technology. This work presents the development of the PON MicroPython 1.0 Framework, specifically designed for microcontrollers compatible with the MicroPython interpreter. During the development process, priority was given to integrating NOP elements with the most common peripherals in microcontrollers, such as General-Purpose Input-Output (GPIO), Analog-to-Digital Converter (ADC) and Pulse Width Modulation (PWM), in addition to a modular mechanism for notification communication between different devices. This communication is facilitated by communication drivers that maintain a high-level programming interface, ensuring usability and flexibility. The results include the development of two practical applications to validate the framework. The first application aims to evaluate the cost of integrating peripherals with NOP elements, both locally and distributed between two microcontrollers. The second application, named Air Quality Monitor, explores the framework's potential in simplifying the distribution of notifications in a more complex scenario involving four microcontrollers and two distinct communication drivers. Both applications demonstrated that the framework streamlined development, reducing the size of the final code while maintaining a high-level programming interface. However, the results also highlighted an increase in latency between actions and in RAM consumption, stemming from the structures used by the framework to provide NOP component abstractions. Additionally, this work details the use of the framework elements, encompassing both those integrated with peripherals and the communication drivers developed. The documentation provided establishes a solid foundation for other developers to adopt and expand the use of the framework in various contexts. Finally, future improvements for the PON MicroPython 1.0 Framework are proposed, focusing on enhancing the communication mechanism.

Keywords: Notification-Oriented Paradigm; Microcontrollers; MicroPython NOP Framework; Distributed systems.

LISTA DE ILUSTRAÇÕES

Figura 1 – Classificação dos paradigmas de programação	21
Figura 2 – Exemplo de um <i>FBE</i> e seus elementos	23
Figura 3 – Fluxo de notificações do PON.....	24
Figura 4 – Diagrama de classes em UML dos componentes base do <i>framework</i>	32
Figura 5 – Camadas do mecanismo de comunicação.....	34
Figura 6 – Diagrama de classes em UML do mecanismo de comunicação	35
Figura 7 – Diagrama de atividades em UML de notificações locais.....	36
Figura 8 – Diagrama de atividades em UML das notificações remotas.....	37
Figura 9 – Diagrama de classes em UML da integração com os periféricos do microcontrolador.....	38
Figura 10 – <i>FBE</i> da aplicação de sensores.....	41
Figura 11 – Diagrama de blocos em SysML da aplicação de sensores.....	54
Figura 12 – Diagrama de blocos em SysML da aplicação de sensores distribuída.	55
Figura 13 – Diagrama de atividades em SysML da aplicação de monitor de qualidade de ar.	56
Figura 14 – Diagrama de blocos em SysML da arquitetura da aplicação AQM distribuída.	57
Figura 15 – Tempo de latência (p99) da aplicação de sensores.....	66
Figura 16 – Uso de memória RAM da aplicação de sensores.....	67
Figura 17 – Conexões entre microcontroladores da aplicação <i>AirQualityMonitor</i>	68
Figura 18 – Tempo de latência da aplicação <i>AirQualityMonitor</i>	70
Figura 19 – Uso de memória RAM da aplicação de AQM.....	72
 Quadro 1 – Propriedades elementares contempladas nas materializações do PON	27
Quadro 2 – Trabalhos realizados em PON no contexto de sistemas distribuídos.	28
Quadro 3 – Parâmetros de construção do <i>Attribute</i>	43
Quadro 4 – Parâmetros de construção do <i>GPIOAttribute</i>	44
Quadro 5 – Parâmetros de construção do <i>ADCAttribute</i>	44
Quadro 6 – Parâmetros de construção do <i>PWMAttribute</i>	45
Quadro 7 – Parâmetros de construção de <i>RemotePremise</i>	46
Quadro 8 – Parâmetros de construção de <i>RemoteCondition</i>	47
Quadro 9 – Parâmetros de construção do <i>UARTDriver</i>	52
Quadro 10 – Parâmetros de construção do <i>UDPDriver</i>	52
Quadro 11 – Propriedades elementares contempladas nas materializações do PON.	58
Quadro 12 – Trabalhos realizados em PON no contexto de sistemas distribuídos.	59

LISTA DE CÓDIGOS

Código 1 – Exemplo de Código com redundâncias.	22
Código 2 – Exemplo de código de <i>FBE</i> com Rule no <i>Framework</i> PON CPP 4.5.	25
Código 3 – Comparação de uma função normal e lambda.....	30
Código 4 – Exemplo da sobrecarga de operadores.	33
Código 5 – Exemplo de uso do <i>GPIOAttribute</i>	39
Código 6 – Exemplo de uso do <i>ADCAttribute</i> e <i>PWMAttribute</i>	40
Código 7 – <i>FBE</i> da aplicação sensores implementada utilizando o <i>framework</i>	41
Código 8 – Leitura e escrita do valor do <i>Attribute</i>	43
Código 9 – Exemplo de criação de <i>Premises</i>	46
Código 10 – Exemplo de uso de <i>RemotePremise</i>	46
Código 11 – Exemplo de criação de uma <i>Condition</i>	47
Código 12 – Exemplo de uso de <i>RemoteCondition</i>	48
Código 13 – Exemplo de criação de <i>Methods</i>	48
Código 14 – Exemplo de criação de uma <i>Instigation</i>	49
Código 15 – Exemplo de criação de uma <i>Action</i>	49
Código 16 – Exemplo de criação de uma <i>Rule</i>	50
Código 17 – Exemplo de criação de uma <i>Rule</i> sem compartilhamento.	51
Código 18 – Exemplo de uso de múltiplos <i>drivers</i> de comunicação.	53
Código 19 – <i>FBE</i> principal da aplicação <i>Sensors</i> local no PON.	61
Código 20 – Código da aplicação <i>Sensors</i> local no POO.	62
Código 21 – <i>FBE</i> principal da aplicação <i>Sensors</i> distribuída em PON.	63
Código 22 – Parte do código da aplicação de sensores em POO.	64

LISTA DE TABELAS

Tabela 1 – Tempos de latência da aplicação de sensores.	65
Tabela 2 – Tempos de latência da aplicação de sensores processados.....	65
Tabela 3 – Linhas de Código por versão da aplicação <i>Sensors</i>	66
Tabela 4 – Tempos de latência da aplicação AQM.	69
Tabela 5 – Tempos de latência da aplicação AQM processados.....	69
Tabela 6 – Linhas por componente da aplicação <i>AirQualityMonitor</i>	71

LISTA DE SIGLAS

ADC	Analog-to-Digital Converter
BLE	Bluetooth Low Energy
FBE	Fact Base Element
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
JSON	JavaScript Object Notation
LED	Light Emitting Diode
PD	Paradigma Declarativo
PF	Paradigma Funcional
PI	Paradigma Imperativo
PON	Paradigma Orientado a Notificações
POO	Paradigma Orientado a Objetos
PWM	Pulse Width Modulation
REPL	Read-Eval-Print-Loop
UART	Universal Asynchronous Receiver-Transmitter
UDP	User Datagram Protocol
UTFPR	Universidade Tecnológica Federal do Paraná

SUMÁRIO

1	INTRODUÇÃO	13
1.1	Motivação	13
1.2	Justificativa	13
1.3	Objetivos.....	14
1.3.1	Objetivo Geral.....	14
1.3.2	Objetivos Específicos	14
1.4	Requisitos	15
1.4.1	Requisitos funcionais.....	15
1.4.2	Requisitos não-funcionais	16
1.5	Método de Desenvolvimento	17
2	FUNDAMENTAÇÃO TEÓRICA.....	19
2.1	Sistemas Embarcados	19
2.2	Paradigmas de Programação.....	20
2.3	Paradigma Orientado a Notificações	23
2.4	Frameworks do Paradigma Orienta a Notificações.....	25
2.5	PON em sistemas embarcados.....	28
2.6	MicroPython	29
3	PROJETO.....	31
3.1	Projeto do <i>framework</i>.....	31
3.1.1	Projeto do mecanismo de notificações	31
3.1.2	Projeto do mecanismo de comunicação	33
3.1.3	Projeto da integração com os periféricos do microcontrolador	37
3.2	Interface do <i>framework</i>.....	41
3.2.1	Fact-Base Element	42
3.2.2	Attribute.....	42
3.2.3	Premise.....	45
3.2.4	Condition.....	47
3.2.5	Method	48
3.2.6	Instigation	49
3.2.7	Action	49
3.2.8	Rule.....	50
3.2.9	Drivers de comunicação	51
3.3	Projeto das aplicações de teste.....	53
3.3.1	Aplicação de sensores	53
3.3.2	Aplicação de monitor de qualidade do ar.....	55
3.4	Considerações	57
4	TESTES E RESULTADOS	60
4.1	Aplicação de sensores	60
4.2	Aplicação de monitor de qualidade do ar.....	67
4.3	Considerações dos resultados.....	72

5	CONCLUSÃO.....	74
5.1	Resultados	74
5.2	Trabalhos futuros	75
5.3	Considerações finais	76

1 INTRODUÇÃO

Este presente capítulo introdutório traz a motivação, justificativa, objetivos e método deste Trabalho de Conclusão de Curso (TCC). Este trabalho de TCC está focado principalmente no estado da técnica do chamado Paradigma Orientado a Notificações (PON), particularmente propondo um *Framework* PON em MicroPython para microcontroladores com ênfase na distribuição de notificações.

1.1 Motivação

O PON busca resolver alguns problemas presentes nos paradigmas consolidados na indústria, como o Paradigma Imperativo (PI) e o Paradigma Declarativo (PD). Os principais problemas tratados são oriundos de uma orientação a percorrimientos e pesquisas, sendo tais problemas as redundâncias estruturais que provocam a avaliação de expressões lógicas repetidas em diferentes partes da aplicação e as redundâncias temporais resultando da reavaliação de expressões lógicas várias vezes sem necessidade, bem como os acoplamentos daí decorrentes. (SIMÃO; STADZISZ, 2009; NEVES, 2020)

O PON resolve estes problemas via um mecanismo de colaboração direta entre entidades notificantes não acopladas e não redundantes, via uma organização ímpar da computação realizada por elas, o que é detalhado no capítulo subsequente. De antemão, entretanto, a materialização dos conceitos propostos pelo PON é realizada por meio de *frameworks* no estado da técnica e pela Tecnologia LingPON no estado da arte. Eis que os resultados alcançados nestes trabalhos são pertinentes e mesmo animadores, ensejando um conjunto de novos trabalhos, como este presente TCC.

1.2 Justificativa

O PON é uma solução ainda emergente para resolver alguns problemas encontrados em paradigmas de programação tradicionais, salientando aqui a Programação Orientada a Objetos (POO) do PI. Atualmente, no acima citado estado da técnica, existem várias materializações dos conceitos propostos pelo PON por meio de *frameworks*. No entanto, estes têm sido projetados para serem usados em sistemas mais complexos e que executam em computadores ou processadores com maior capacidade de computação (NEVES, 2020; FIGUEIREDO, 2022). Isto

justamente dificulta os seus usos em microcontroladores, nos quais os recursos computacionais disponíveis são mais modestos (RASPERRY, 2020).

Em suma, nenhum dos *frameworks* do PON utiliza as suas características especificamente para microcontroladores, particularmente o desacoplamento implícito das entidades, para poder abstrair a interação entre diferentes microcontroladores utilizando alguma interface de comunicação. Ao ensejo, esse tipo de interface costuma ser um desafio considerável para os iniciantes em sistemas embarcados. Desta forma, um *framework* PON enxuto, orientado especificamente a dispositivos embarcados e focado na abstração dos mecanismos de comunicação permitiria aos desenvolvedores de aplicações embarcadas ter uma maior eficiência no momento de implementar suas soluções.

1.3 Objetivos

Esta presente seção destaca o objetivo geral, bem como os daí derivados objetivos específicos.

1.3.1 Objetivo Geral

Desenvolver um novo *framework* do PON em MicroPython que permita o desenvolvimento de *software* neste paradigma para microcontroladores, com foco na abstração dos mecanismos de comunicação e integração com os seus principais periféricos.

De antemão, tal *framework* para PON em microcontroladores é denominado de *Framework* PON MicroPython 1.0.

1.3.2 Objetivos Específicos

Com base ao objetivo geral, foram determinados os seguintes objetivos específicos:

- Desenvolver um novo *framework* PON em MicroPython com seus componentes principais denominado *Framework* PON MicroPython 1.0.
- Desenvolver um mecanismo para comunicação de notificações modular entre os elementos *Attribute*, *Premise* e *Condition* do PON integrado ao *Framework* PON MicroPython 1.0.

- Desenvolver uma aplicação que utilize pelo menos três microcontroladores Raspberry Pi Pico W para demonstrar o funcionamento do *Framework* PON MicroPython 1.0 e o protocolo de comunicação.
- Desenvolver a documentação de uso do *framework*.

1.4 Requisitos

Os requisitos deste trabalho foram estabelecidos com base no objetivo geral e nos objetivos específicos definidos. Os requisitos funcionais, que descrevem as funcionalidades que Framework PON MicroPython 1.0 deve apresentar, estão detalhados na seção 1.4.1. Por sua vez, os requisitos não-funcionais, que especificam características de compatibilidade, usabilidade, entre outros, estão apresentados na seção 1.4.2.

1.4.1 Requisitos funcionais

Nesta seção, são apresentados os requisitos funcionais que o Framework PON MicroPython 1.0 deve implementar para atender ao objetivo geral e aos objetivos específicos. Tais requisitos são:

- **RF-01:** O Framework PON MicroPython 1.0 deve permitir a criação de *Attributes*, *Premises*, *Conditions*, *Rules*, *Actions*, *Instigations*, *Methods* e *FBEs*.
- **RF-02:** O Framework PON MicroPython 1.0 deve permitir associar o valor de um *Attribute* ao valor de um pino ADC.
- **RF-03:** O Framework PON MicroPython 1.0 deve permitir associar o valor de um *Attribute* ao estado lógico de um pino GPIO.
- **RF-04:** O Framework PON MicroPython 1.0 deve permitir associar o estado lógico de um pino GPIO ao valor de um *Attribute*.
- **RF-05:** O Framework PON MicroPython 1.0 deve permitir associar tanto o ciclo de trabalho como a frequência do um gerador PWM a um *Attribute* específico.
- **RF-06:** O Framework PON MicroPython 1.0 deve permitir a comunicação de notificações entre os elementos locais.
- **RF-07:** O Framework PON MicroPython 1.0 deve permitir a comunicação de notificações entre diferentes microcontroladores.

- **RF-08:** O Framework PON MicroPython 1.0 deve identificar os elementos locais dos distribuídos por meio de um identificador (id).
- **RF-09:** O Framework PON MicroPython 1.0 deve distribuir os elementos com id maior que zero.
- **RF-10:** O Framework PON MicroPython 1.0 deve permitir a distribuição dos elementos pelas interfaces UART do microcontrolador.
- **RF-11:** O Framework PON MicroPython 1.0 deve permitir a distribuição dos elementos por meio do protocolo UDP.
- **RF-12:** O Framework PON MicroPython 1.0 deve permitir a criação de *Premises* utilizando os operadores lógicos de comparação (`==`, `!=`, `>`, `>=`, `<=`, `<`), tanto entre instâncias de dois *Attributes* tanto quanto de *Attribute* para com um valor constante do mesmo tipo que o *Attribute*.
- **RF-13:** O Framework PON MicroPython 1.0 deve permitir a criação de *Conditions* utilizando os operadores de conjunção (`&`) e disjunção (`||`), tanto entre instâncias de diferentes *Premises* quanto *Conditions*.

1.4.2 Requisitos não-funcionais

Nesta seção, por sua vez, são apresentados os requisitos não-funcionais que o Framework PON MicroPython 1.0 deve implementar para atender ao objetivo geral e aos objetivos específicos. Tais requisitos são:

- **RNF-01:** O Framework PON MicroPython 1.0 deve ser implementado na linguagem de programação MicroPython.
- **RNF-02:** O Framework PON MicroPython 1.0 deve ser compatível com o microcontrolador RP2040.
- **RNF-03:** O Framework PON MicroPython 1.0 deve implementar a cadeia de notificações utilizando o padrão de programação *Observer-Observable*.
- **RNF-04:** O id dos elementos do Framework PON MicroPython 1.0 deve ser um número inteiro sem sinal de 32 bits.
- **RNF-05:** A serialização dos elementos distribuídos do Framework PON MicroPython 1.0 deve ser realizada utilizando o formato *JavaScript Object Notation* (JSON).

- **RNF-06:** O Framework PON MicroPython 1.0 deve permitir o uso de todos os seus elementos pela importação do pacote `pon`.
- **RNF-07:** O Framework PON MicroPython 1.0 deve ter uma interface unificada para todos os meios de transmissão de notificações.
- **RNF-08:** Os meios de comunicação de notificações do Framework PON MicroPython 1.0 devem ser totalmente desacoplados um dos outros.
- **RNF-09:** Os FBEs do Framework PON MicroPython 1.0 devem ser representados por classes do MicroPython.
- **RNF-10:** Os *Methods* do Framework PON MicroPython 1.0 devem ser implementados por métodos do MicroPython.
- **RNF-11:** A documentação do Framework PON MicroPython 1.0 deve detalhar todos os parâmetros para instanciar cada elemento do PON.

1.5 Método de Desenvolvimento

O trabalho foi estruturado em seis etapas principais: análise e estudo, projeto, implementação, testes e resultados, documentação e conclusão.

Na primeira etapa, a análise e estudo, serão avaliadas as diferentes linguagens (C++ e MicroPython) e plataformas viáveis para a implementação do *framework* (Raspberry Pi Pico, ESP32, etc.). Também será realizada uma investigação sobre sistemas de comunicação existentes, a fim de definir o protocolo mais adequado para o mecanismo de comunicação a ser desenvolvido.

A segunda etapa, referente ao projeto, consiste no desenvolvimento do *design* do *framework* e do mecanismo de comunicação. Nessa fase, serão criados diagramas e especificações detalhadas, visando facilitar a implementação posterior. Na etapa de implementação, os diagramas e especificações elaborados anteriormente serão utilizados para a construção do *framework* na linguagem selecionada.

Durante a fase de testes e resultados, serão implementadas duas aplicações principais para validar o *framework* proposto, cada uma explorando diferentes cenários. A primeira aplicação, denominada "Sensores", envolve a interação com os periféricos de um único microcontrolador. Já a segunda, uma versão distribuída do *AirQualityMonitor* (BABU, 2022), utiliza três ou mais microcontroladores para testar o *framework* em cenários distribuídos.

A quinta etapa envolve a criação de um manual de uso, fornecendo uma explicação clara e prática sobre como utilizar cada elemento do *framework*. Por fim, na etapa de conclusão, será escrito o relatório final do trabalho e preparada a apresentação conclusiva do projeto.

2 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica deste trabalho é composta por seis tópicos principais, que fornecem o contexto necessário para o desenvolvimento do *framework* e do mecanismo de comunicação propostos. Em suma, será abordado o conceito de sistemas embarcados (seção 2.1), paradigmas de programação (seção 2.2), o próprio paradigma orientado a notificações (seção 2.3) e seus *frameworks* (seção 2.4). Na seção 2.5 são apresentados conceitos relativos ao PON aplicado aos sistemas embarcados. Finalmente, na seção 2.6 a linguagem MicroPython é brevemente apresentada.

2.1 Sistemas Embarcados

Os sistemas embarcados são dispositivos computacionais projetados para executar funções específicas dentro de um sistema maior, muitas vezes operando em tempo real e sob restrições de hardware e energia. Segundo Lee e Seshia (2017), esses sistemas são caracterizados pela integração entre software e hardware, resultando em soluções otimizadas para aplicações específicas, como dispositivos móveis, automação industrial e veículos autônomos.

Para possibilitar a interação com sensores e atuadores, os sistemas embarcados contam com diversos periféricos integrados, como GPIO, ADC, PWM, UART e I2C. A seguir, são abordadas as principais características desses componentes fundamentais:

- **GPIO:** são fundamentais na comunicação entre microcontroladores e periféricos externos, permitindo controle e monitoramento de sinais digitais (RENESAS, 2025). Como entradas, os GPIOs podem ler estados lógicos de dispositivos externos, como botões ou sensores digitais. Como saídas, podem controlar dispositivos como LEDs, relés ou outros componentes eletrônicos.
- **ADC:** são dispositivos essenciais para a conversão de sinais analógicos em digitais, permitindo que microcontroladores e outros dispositivos digitais processem informações do mundo real, como temperatura, tensão e som. (TEXAS INSTRUMENTS, 2025).

- **PWM:** é uma técnica utilizada para controlar a potência entregue a dispositivos eletrônicos, variando a relação entre tempo ligado e desligado de um sinal periódico. Essa abordagem é amplamente aplicada no controle de motores, na regulação de brilho de LEDs e na conversão de energia em fontes chaveadas (TEXAS INSTRUMENTS, 2025).
- **UART:** é um protocolo de comunicação serial amplamente utilizado para a transmissão de dados entre dispositivos eletrônicos. Diferente de protocolos síncronos, o UART não requer um sinal de *clock* compartilhado entre o transmissor e o receptor, utilizando apenas linhas de transmissão e recepção para comunicação. Esse método é comumente empregado em microcontroladores, sensores e módulos de comunicação (TEXAS INSTRUMENTS, 2025).
- **I²C:** é uma tecnologia de comunicação serial que utiliza apenas duas linhas para transmitir e receber dados: uma para o sinal de *clock* e outra para os dados. Ele é amplamente usado em sistemas embarcados, devido à sua simplicidade e capacidade de conectar vários dispositivos em um único barramento (TEXAS INSTRUMENTS, 2025).

Os sistemas embarcados são tradicionalmente programados em linguagens C/C++. Não obstante, há outras linguagens de programação que emergem neste contexto, sendo que aqui é destacado o MicroPython. Essas linguagens de programação citadas fazem parte do paradigma imperativo. Justamente, a temática de paradigmas para programação em geral é tratada na seção que segue.

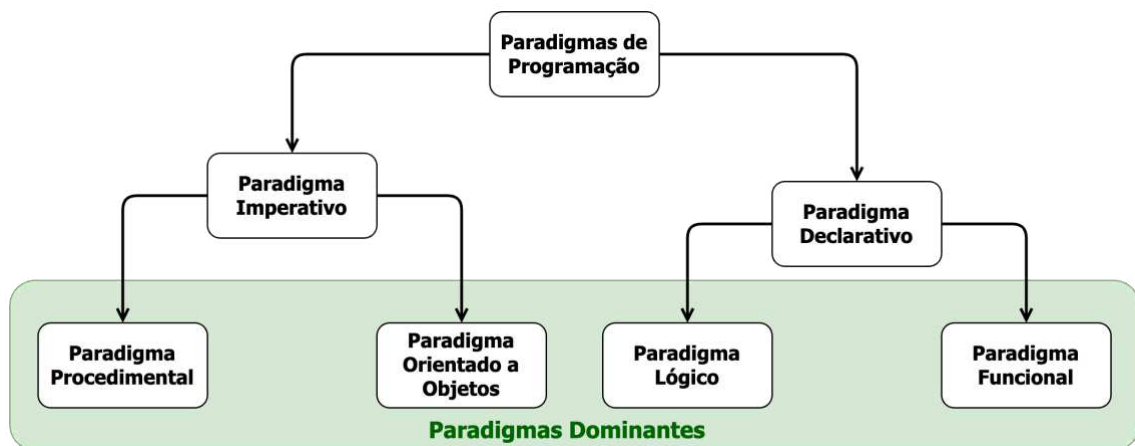
2.2 Paradigmas de Programação

Um paradigma de programação é compreendido como um modelo ou método para o desenvolvimento de programas computacionais, fundamentado em estruturas linguístico-matemáticas e em um conjunto específico de princípios. Cada paradigma é projetado para abordar determinados problemas, guiando-se por seus modelos e princípios subjacentes. Dessa forma, cada paradigma possui um conjunto de regras que pode torná-lo mais ou menos adequado para a resolução de problemas específicos (ROY, 2012).

Os principais paradigmas de programação podem ser agrupados em quatro categorias: o Paradigma Procedimental (PP), o Paradigma Funcional (PF), o Paradigma Orientado a Objetos (POO) e o Paradigma Lógico (PL) (SCOTT, 2000; WATT, 2004; BROOKSHEAR, 2006). Esses paradigmas são frequentemente denominados como Paradigmas Dominantes. Além disso, considera-se em termos gerais que o PP e o POO estão inseridos em um paradigma mais amplo, conhecido como Paradigma Imperativo (PI), enquanto o PF e o PL fazem parte principalmente de outro grande grupo, o Paradigma Declarativo (PD) (GABBRIELLI; MARTINI, 2010).

Dito de outra forma, os paradigmas dominantes podem ser classificados como subconjuntos de dois paradigmas maiores: o Paradigma Imperativo (PI) e o Paradigma Declarativo (PD) (BANASZEWSKI, 2009). Essa relação é representada hierarquicamente, como ilustrado na Figura 1. Apesar de ser uma classificação útil como visão geral, ela não é perfeita, pois os paradigmas compartilham características entre si e frequentemente se aproximam mais de um paradigma dominante do que de outro. Em resumo, essa organização hierárquica facilita as comparações, destacando as características comuns entre os paradigmas (RONSZCKA, 2019).

Figura 1 – Classificação dos paradigmas de programação



Fonte: Adaptado de Ronszcka (2019)

Os paradigmas atualmente utilizados, como o PI, frequentemente geram redundâncias de natureza temporal (ou seja, reavaliações desnecessárias ao longo do tempo) e estrutural (repetições de código), especialmente nas análises lógico-causais. Esse fenômeno resulta em desperdício de recursos computacionais ao longo do tempo (SIMÃO; STADZISZ, 2009). O Código 1 ilustra um exemplo clássico de implementação do paradigma PI em C/C++, onde as redundâncias temporais e estruturais tornam-se evidentes.

Código 1 – Exemplo de Código com redundâncias.

```

bool sensorA_ActiveAlarm = true;
bool sensorB_ActiveAlarm = false;
bool sensorB_PresenceDetected = true;
// . . .
while (sensorB_PresenceDetected == true) {
    if (sensorA_ActiveAlarm == true) {
        sensorB_ActiveAlarm = true;
    }
    if ((sensorB_PresenceDetected == true) && (sensorB_ActiveAlarm == false)) {
        sensorB_ActiveAlarm = true;
    }
    // n code lines . . .
    if ((sensorB_PresenceDetected == true) &&
        (sensorB_ActiveAlarm == true) &&
        (sensorA_PresenceDetected == true)) {
        sensorA_ActiveAlarm = true;
    }
}

```

Fonte: Adaptado de Oshiro (2021)

No trecho destacado em vermelho do Código 1, observa-se uma avaliação lógica que se repete a cada ciclo iterativo, mesmo que não haja alterações em relação às avaliações anteriores, caracterizando uma redundância temporal. Já as redundâncias estruturais podem ser identificadas nos trechos destacados em verde, onde a primeira avaliação lógica dessas duas expressões causais é estruturalmente repetida no código, sendo executada duas vezes em cada ciclo. Além disso, essas redundâncias podem estar dispersas no código, separadas por várias linhas, o que dificulta sua identificação e mitigação, mesmo para desenvolvedores experientes, tornando inviável tratá-las em códigos de grande escala. De forma cumulativa, tais redundâncias podem resultar em desperdícios significativos de recursos de processamento (PAN et al., 1998).

Além deste exemplo em PI, outros subparadigmas ou técnicas associadas ao PI, PD ou à interseção entre ambos apresentam problemas semelhantes, como discutido em RAYMOND (2003), BANERJEE et al. (1995), OSHIRO et al. (2021) e RONSZCKA et al. (2017). Essas redundâncias são também observadas em paradigmas como o procedimental, orientado a objetos e funcional, que compartilham características semelhantes. Por outro lado, a programação lógico-declarativa, embora opere em um nível mais abstrato, é implementada sobre o PI e, portanto, não está isenta de redundâncias. Técnicas como a orientação a eventos, apesar de

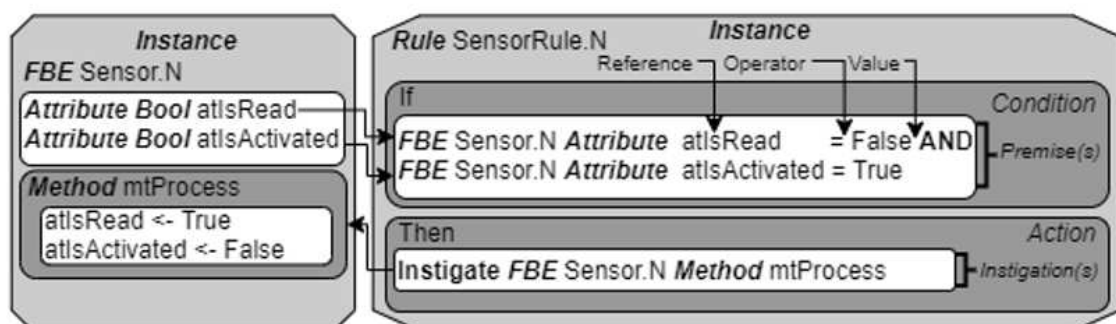
contribuir para mitigar esses problemas, também não conseguem eliminá-los por completo (RONSZCKA, 2019; OSHIRO et al., 2021).

Adicionalmente, os paradigmas vigentes frequentemente geram módulos acoplados devido às redundâncias inerentes. Esse acoplamento exige conhecimento técnico avançado em programação e modelagem para criar módulos parcialmente desacoplados, visando organização, reaproveitamento e suporte ao processamento paralelo ou distribuído. No entanto, mesmo com esses esforços técnicos, o desacoplamento obtido raramente atinge um nível de granularidade suficiente para otimizar o balanceamento de carga em sistemas paralelos ou distribuídos (OSHIRO et al., 2021). Nesse contexto, o paradigma PON surge como uma alternativa que busca, entre outras metas, eliminar redundâncias, contribuindo para a mitigação desses desafios.

2.3 Paradigma Orientado a Notificações

O PON é um paradigma emergente no qual há a separação de entidades de processamento em uma parte facto-execucional (ex. estados de elementos e execução de procedimentos/ações que alteram seus estados) e uma parte lógico-causal (i.e., decisão de quais procedimentos/ações executar) que colaboram por notificações pontuais. A parte facto-execucional é representada por *Fact Base Elements (FBEs)* com seus *Attributes* e *Methods*, enquanto a parte lógico-causal é representada por *Rules* com suas *Conditions-Premises* e *Actions-Instigations*, sendo que se pode observar um exemplo destas entidades na Figura 2. (SIMÃO; STADZISZ, 2009; BANASZEWSKI, 2009)

Figura 2 – Exemplo de um *FBE* e seus elementos

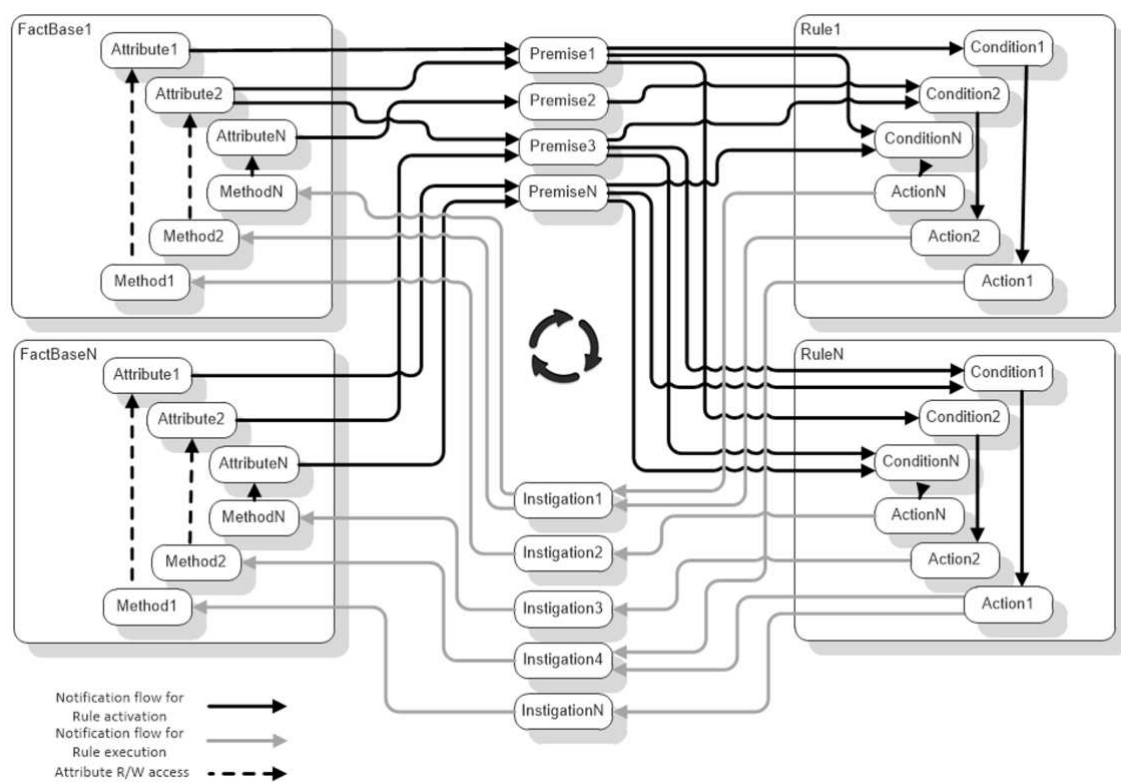


Fonte: Adaptado de Neves (2021)

Inicialmente, tem-se os *Fact Base Elements (FBEs)* que contêm os *Attributes* e *Methods*. Cada *Attribute* representa uma propriedade de um *FBE*, enquanto cada

Method representa uma funcionalidade dele. Nas *Rules*, por sua vez, as *Premises* são encarregadas de realizar a avaliação de uma expressão lógica contendo *Attributes*, ao passo que as *Conditions* são encarregadas de avaliar uma expressão lógica contendo *Premises*. Onde, cada *Rule* aprovada executa sua *Action* que se encarrega de executar suas *Instigations* associadas. Finalmente, cada *Instigation* executa os seus *Methods* associados. Particularmente, toda colaboração entre estas entidades é realizada exclusivamente por meio de notificações no termos indicados na Figura 3.(SIMÃO; STADZISZ, 2009; NEVES, 2021).

Figura 3 – Fluxo de notificações do PON



Fonte: Linhares (2015)

Todas essas entidades do PON colaboram por meio de uma elegante e precisa cadeia de notificações. A cadeia de notificações do PON e todos seus elementos podem ser observados na Figura 3, e detalhada a partir dela. Em suma, na mudança de estado pertinente, cada *Attribute* notifica precisamente apenas as *Premises* pertinentes. Cada *Premise* que mude de estado lógico em função da notificação recebida, notifica apenas as *Conditions* pertinentes. Cada *Condition* que mude de estado lógico em função da notificação recebida, notifica sua *Rule*. Após receber notificação de todas suas *Conditions*, a *Rule* é considerada aprovada. Cada *Rule* aprovada, uma vez resolvido qualquer conflito (e.g., via prioridade), notifica sua

Action, a qual notifica suas *Instigations*. Cada *Instigation*, por sua vez, notifica seus *Methods*. Por fim, cada *Method* pode alterar estados de um conjunto de *Attributes*, realimentando assim o ciclo de inferência. Em tempo, naturalmente, as conexões de notificação são estabelecidas em tempo de construção das entidades do PON (SIMÃO; STADZISZ, 2009).

2.4 Frameworks do Paradigma Orienta a Notificações

Há um conjunto de *frameworks* em PON elaborados com linguagens de programação conhecidas (e.g., C++, Java, C# e Erlang/Elixir) que permitem estas trabalharem de forma orientada a notificações. Isto é implementado por meio de uma camada de abstração que permite a declaração dos elementos do PON, como é apresentado no exemplo do Código 2.

Código 2 – Exemplo de código de FBE com Rule no Framework PON CPP 4.5.

```
struct Sensor : NOP::FBE
{
    NOP::SharedAttribute<bool> atIsRead{NOP::BuildAttribute(false)};
    NOP::SharedAttribute<bool> atIsActivated{NOP::BuildAttribute(false)};

    NOP::SharedRule rlSensor;

    void activate() const {
        atIsActivated->SetValue(true);
        atIsRead->SetValue(false);
    }
    void read() const { atIsRead->SetValue(true); }
    void deactivate() const { atIsActivated->SetValue(false); }

    Sensor()
    {
        RULE(&rlSensor);
        LCONDITION();
        CEXP(atIsActivated == true) AND CEXP(atIsRead == false);
        END_CONDITION;
        ACTION();
        INSTIGATE(METHOD(this->read(); this->deactivate()));
        END_ACTION;
        END_RULE;
    }
};
```

Fonte: O autor.

O *framework* inicial foi proposto por Simão (2007) em C++, com base no *framework* de Controle Orientado a Notificações (SIMÃO, 2001; Simão & Stadzisz, 2008). Em 2009, Banaszewski (2009) apresentou o *Framework* PON CPP 1.0, oferecendo maior estabilidade e empacotamento para classes. Ronszcka (2012) e

Valença (2012) introduziram o *Framework* PON CPP 2.0, utilizando padrões de projeto e códigos mais otimizados para facilitar a programação e melhorar o desempenho. Por fim, o *Framework* PON C++ 3.0, incorporou funcionalidades *multicore* (BELMONTE et al., 2012; SCHÜTZ, 2019). Atualmente, os últimos *frameworks* para a linguagem de programação C++ são o *Framework* PON CPP 4.0 (FWPONCPP 4.0) (NEVES, 2021), o *Framework* PON CPP 4.5 (FWPONCPP 4.5) (KUNNEN, 2022), e sua versão orientada a dispositivos de *Internet Of Things* (IoT) o *Framework* PON CPP 4.0 IoT (FW PON CPP 4.0 IoT) (FIGUEIREDO, 2022), que será explicada com mais detalhes posteriormente.

Além dos *frameworks* em C++, também foram desenvolvidas implementações em outras linguagens de programação. Os *frameworks* em C# e Java surgiram como adaptações do *Framework* PON CPP 1.0 para essas linguagens (HENZEN, 2015). Posteriormente, Oliveira (2019) adaptou e evoluiu o *Framework* C# de Henzen (2015), resultando no *Framework* PON C# IoT. Adicionalmente, foram criados dois *frameworks* alinhados a modelos de atores: um em Elixir/Erlang (NEGRINI et al., 2019; NEGRINI, 2019) e outro em Akka.Net (MARTINI et al., 2018).

No Quadro 1 são apresentadas as propriedades elementares implementadas por cada *framework* mencionado. Onde se pode acompanhar a evolução dos diferentes graus de implementação de cada propriedade.

Quadro 1 – Propriedades elementares contempladas nas materializações do PON

Framework	Programação em alto nível	Paralelismo via desacoplamento	Distribuição via desacoplamento	Código não redundante
FW PON CPP 1.0 (BANASZEWSKI, 2009)	parcialmente	-	-	-
FW PON CPP 2.0 (RONSZCKA, 2012 ; VALENÇA, 2012)	Parcialmente via <i>wizard</i>	-	-	parcialmente
FW PON CPP 3.0 (BELMONTE et al, 2012 ; SCHÜTZ, 2019)	parcialmente	parcialmente	-	-
FW PON CPP 4.0 (NEVES, 2021)	sim	sim	-	sim
FW PON CPP 4.5 (KUNNEN, 2022)	sim	sim	-	sim
FW PON CPP 4.0 IoT (FIGUEIREDO, 2022)	sim	sim	sim	sim
FW PON Java/C# (HENZEN, 2015)	parcialmente	-	-	parcialmente
FW PON Erlang/Elixir (NEGRINI et al. 2019)	parcialmente	sim	suportado, porém não validado	-
FW PON Akka.Net (MARTINI et al. 2019)	Parcialmente	sim	suportado, porém não validado	-
FW PON C# IoT (OLIVEIRA, 2019)	parcialmente	sim	sim	-

Fonte: Adaptado de FIGUEIREDO (2022).

O Quadro 2 apresenta um resumo das principais contribuições para o desenvolvimento de tecnologia no contexto do PON. Vale destacar que as contribuições dos trabalhos não se limitam às informações resumidas no quadro, que foram organizadas por Figueiredo (2022) com foco específico em sistemas distribuídos e IoT. Além disso, ressalta-se que, embora os estudos realizados com os *frameworks* Akka.Net (MARTINI et al., 2018) e Erlang/Elixir (NEGRINI et al., 2019) ofereçam suporte à distribuição de elementos como uma funcionalidade herdada das linguagens ou bibliotecas empregadas, essa característica não foi explorada nos trabalhos analisados. Por essa razão, tais características foram classificadas como "Não Avaliadas" (N/A).

Quadro 2 – Trabalhos realizados em PON no contexto de sistemas distribuídos.

Framework	Distribuição	Paralelismo	Entidades distribuídas	Protocolo de transporte	Protocolo de aplicação
FW PON CPP 2.0 + PONIP (RONSZCKA, 2012 ; VALENÇA, 2012)	Sim	Não	Attributes Premises	UDP/TCP	PONIP
FW PON CPP 3.0 + LobeNOI + PONIP (BELMONTE et al, 2012 ; SCHÜTZ, 2019)	Sim	Sim	Attributes Premises	UDP/TCP	PONIP
FW PON Erlang/Elixir (NEGRINI et al., 2019)	Potencialmente	Sim	N/A	N/A	N/A
FW PON Akka.Net (MARTINI et al. 2019)	Potencialmente	Sim	N/A	N/A	N/A
FW PON C# IoT (OLIVEIRA, 2019)	Sim	Sim	Atributes	UDP/TCP	HTTP Próprio
FW PON Java + Attributes Distribuídos (BARRETO et al. 2018)	Sim	Não	Attributes	TCP	Próprio
MicroPON em IoT (MAMANN et al. 2021)	Sim	Não	Atributes	TCP	Próprio
FW PON CPP 4.0 IoT (FIGUEIREDO, 2022)	Sim	Sim	Todos	TCP	MQTT

Fonte: Adaptado de FIGUEIREDO (2022).

Pode-se observar que, embora alguns trabalhos já tenham explorado o PON no contexto de sistemas distribuídos e IoT, cada um segue uma abordagem distinta, o que pode dificultar sua padronização, uso e adoção nesses sistemas. Além disso, nenhum dos *frameworks* analisados permite a distribuição de todos os componentes do PON, com exceção do *Framework* PON CPP 4.0 IoT, que se destaca como o único a implementar a distribuição completa de todos os elementos do PON.

2.5 PON em sistemas embarcados

O uso de *frameworks* do PON em sistemas embarcados, caracterizados pela ausência de sistemas operacionais tradicionais (Unix/Linux/Windows) e por recursos computacionais limitados, ainda é pouco explorado. Essa lacuna representa um obstáculo significativo para a adoção do PON nesses sistemas, pois adiciona um nível adicional de complexidade, especialmente na interação com os periféricos do microcontrolador, além de diminuir a abstração do PON.

Além disso, uma limitação comum a todos os *frameworks* existentes é a implementação da distribuição dos elementos do PON exclusivamente por meio de protocolos de rede. Isso inviabiliza a utilização do PON em cenários onde a distribuição precisa ocorrer por outros meios de comunicação, como interfaces *Universal Asynchronous Receiver/Transmitter* (UART) ou *Inter-Integrated Circuit* (I²C), que são amplamente utilizadas em sistemas embarcados. Essa restrição reduz a flexibilidade e a aplicabilidade do PON em ambientes com restrições de hardware ou requisitos específicos de comunicação, destacando a necessidade de novas abordagens que considerem a heterogeneidade dos sistemas embarcados.

2.6 MicroPython

No início dos anos 2010, o programador e físico britânico Damien George percebeu uma lacuna no mundo da programação embarcada. Enquanto linguagens como C e Assembly dominavam os microcontroladores, faltava uma opção mais acessível e moderna para desenvolvedores e entusiastas. Inspirado na simplicidade e no poder do Python, Damien decidiu criar uma versão enxuta da linguagem, capaz de rodar em hardware de baixo custo e recursos limitados. Para financiar o projeto, Damien lançou uma campanha no Kickstarter em dezembro de 2013, propondo não apenas o software, mas também um hardware dedicado: a PyBoard, uma placa de desenvolvimento otimizada para rodar MicroPython (DAMIEN et al, 2023).

Com esse impulso, o MicroPython foi oficialmente lançado em 2014, trazendo uma implementação enxuta e eficiente da linguagem de programação Python 3, que inclui um subconjunto reduzido da biblioteca padrão do Python e é otimizada para operar em microcontroladores e em ambientes com recursos limitados (DAMIEN et al, 2023). Essa tecnologia consiste em um compilador e ambiente de execução completo, projetado para executar diretamente sobre o hardware, sem a necessidade de sistemas operacionais convencionais. Uma de suas características mais notáveis é o *Read-Eval-Print-Loop* (REPL), um terminal interativo que permite executar comandos imediatamente, facilitando o desenvolvimento e depuração de aplicações. O REPL oferece funcionalidades como histórico de comandos, autocompletar com tabulação, indentação automática e um modo de colagem de código, proporcionando uma experiência fluida ao usuário.

O MicroPython foi desenvolvido com o objetivo de ser o mais compatível possível com o Python tradicional (também conhecido como CPython). Assim,

conhecimentos em Python são diretamente aplicáveis ao uso do MicroPython. Além disso, explorar o MicroPython em ambientes de hardware restrito contribui para uma compreensão mais aprofundada da linguagem Python como um todo.

Além de implementar um conjunto essencial das bibliotecas principais do Python, o MicroPython inclui módulos específicos, como o módulo "machine", que permite o acesso direto ao hardware de baixo nível. Essa funcionalidade é especialmente útil em sistemas embarcados, onde a interação com sensores, atuadores e interfaces de comunicação, como GPIO, UART e I2C, é fundamental para o desenvolvimento de aplicações.

Em suma, o MicroPython se destaca como uma ferramenta poderosa para a programação em sistemas embarcados, unindo a simplicidade do Python com a capacidade de operar em dispositivos com recursos limitados. Outro destaque, no MicroPython, assim como no Python tradicional, são as funções lambda, as quais se constituem em funções anônimas e curtas, definidas de maneira compacta. Elas são úteis quando precisamos de uma função pequena e de curta duração, sem a necessidade de definir um nome explícito. No Código 3, é apresentada a declaração da função "soma" nas linhas 1 e 2, logo, na linha 6 é apresentada a mesma funcionalidade em uma função lambda.

Código 3 – Comparação de uma função normal e lambda.

```
1 | def soma(a, b):  
2 |     return a + b  
3 |  
4 | print(soma(3, 5)) # Saída: 8  
5 |  
6 | soma_lambda = lambda a, b: a + b  
7 |  
9 | print(soma_lambda(3, 5)) # Saída: 8
```

Fonte: O autor.

3 PROJETO

Nesta seção será apresentado o processo de definição e implementação do próprio *Framework PON MicroPython 1.0* proposto assim como o projeto das aplicações de testes.

3.1 Projeto do *framework*

O projeto do *framework* foi estruturado em duas partes principais: projeto do mecanismo de notificações e projeto do mecanismo de comunicação. A primeira parte abrange o desenvolvimento dos componentes fundamentais do *framework*, incluindo o mecanismo interno de notificações e sua interação com os pinos digitais do GPIO e outros periféricos do microcontrolador. Já a segunda parte foca no mecanismo de comunicação, responsável por viabilizar a troca de notificações entre diferentes microcontroladores.

3.1.1 Projeto do mecanismo de notificações

Para a escolha da linguagem de programação a ser utilizada no desenvolvimento do *framework*, foram consideradas duas opções principais: C++ e MicroPython. Optou-se pelo MicroPython, uma alternativa relativamente recente no domínio de programação para microcontroladores, que oferece um nível de abstração mais alto em comparação ao C++, mantendo o suporte para recursos como polimorfismo, herança múltipla e sobrecarga de operadores. Esses recursos são fundamentais para a implementação do *framework* e tornam possível sua execução em qualquer microcontrolador compatível com o interpretador MicroPython, ampliando sua flexibilidade e aplicabilidade em diversos contextos de hardware. Apesar de sua diferença em termos de desempenho em relação a linguagens como C++, o MicroPython se destacou pela sua elevada portabilidade entre diferentes microcontroladores. Isto permitira o *framework* ser executado em microcontroladores que sejam compatíveis com uma versão do *runtime* do MicroPython, como a Raspberry Pi Pico W (RASPBERRY, 2023)

Uma vez definida a plataforma a ser utilizada para o desenvolvimento, foi realizado o diagrama de classes dos componentes base do *framework*, que pode ser vista na Figura 4. Como pode-se observar no diagrama de classes, a base de todos os componentes lógico-causal são o *Notifier-Notifiable*, que é baseado no padrão de

do microcontrolador e para a integração do mecanismo de comunicação de notificações.

Para garantir uma sintaxe clara e concisa no uso do *framework*, foram sobrecarregados os operadores de comparação (==, !=, >, >=, <, <=) da classe *Attribute*, permitindo que retornem instâncias da classe *Premise* com base na operação realizada. Essa abordagem foi implementada tanto para operações entre *Attribute-Attribute* quanto entre *Attribute-valor*, eliminando a necessidade de criar manualmente instâncias de *Attribute* para cada comparação dentro de uma *Premise*, que pode ser verificado no Código 4.

Código 4 – Exemplo da sobrecarga de operadores.

```
atA = Attribute (10)
atB = Attribute (20)
atC = Attribute (30)

prA = atB > 10
prB = atA != atC

cnA = prA & prB
cnB = prA | prB
```

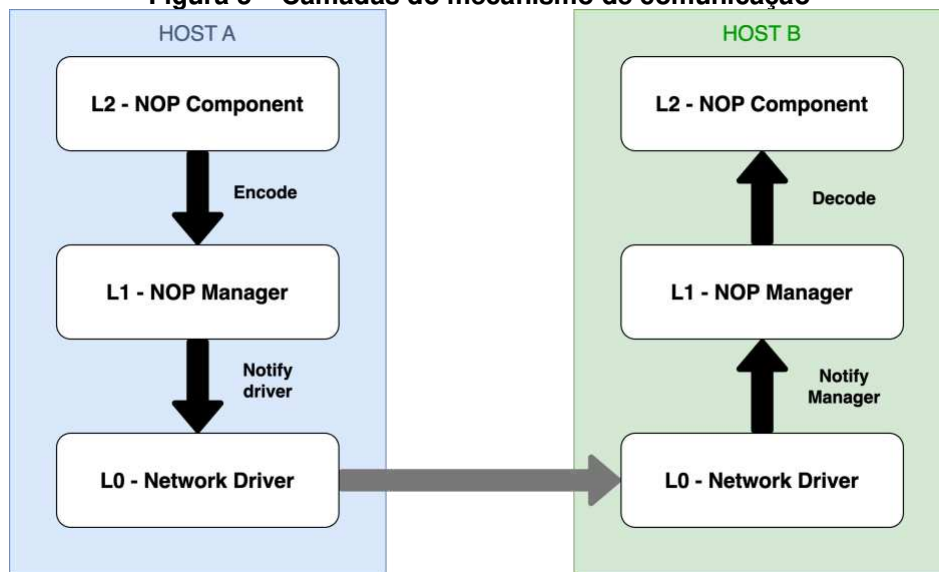
Fonte: O autor.

Além disso, foram sobrecarregados os operadores lógicos “&” e “|” para a classe *Premise*, possibilitando a criação de instâncias da classe *Condition* que representam operações lógicas do tipo *AND* e *OR*, respectivamente. Essa funcionalidade simplifica a construção de condições compostas, mantendo o código mais expressivo e legível.

3.1.2 Projeto do mecanismo de comunicação

No projeto do mecanismo de comunicação foi considerada a modularidade e facilidade de adicionar novos meios de transmissão como características fundamentais. Desta forma optou-se por um modelo de camadas, como pode ser observado na Figura 5, no qual foram definidas três camadas: o próprio elemento (*NOP Component*), o gerenciador (*NOP Manager*) e o *driver* de comunicação.

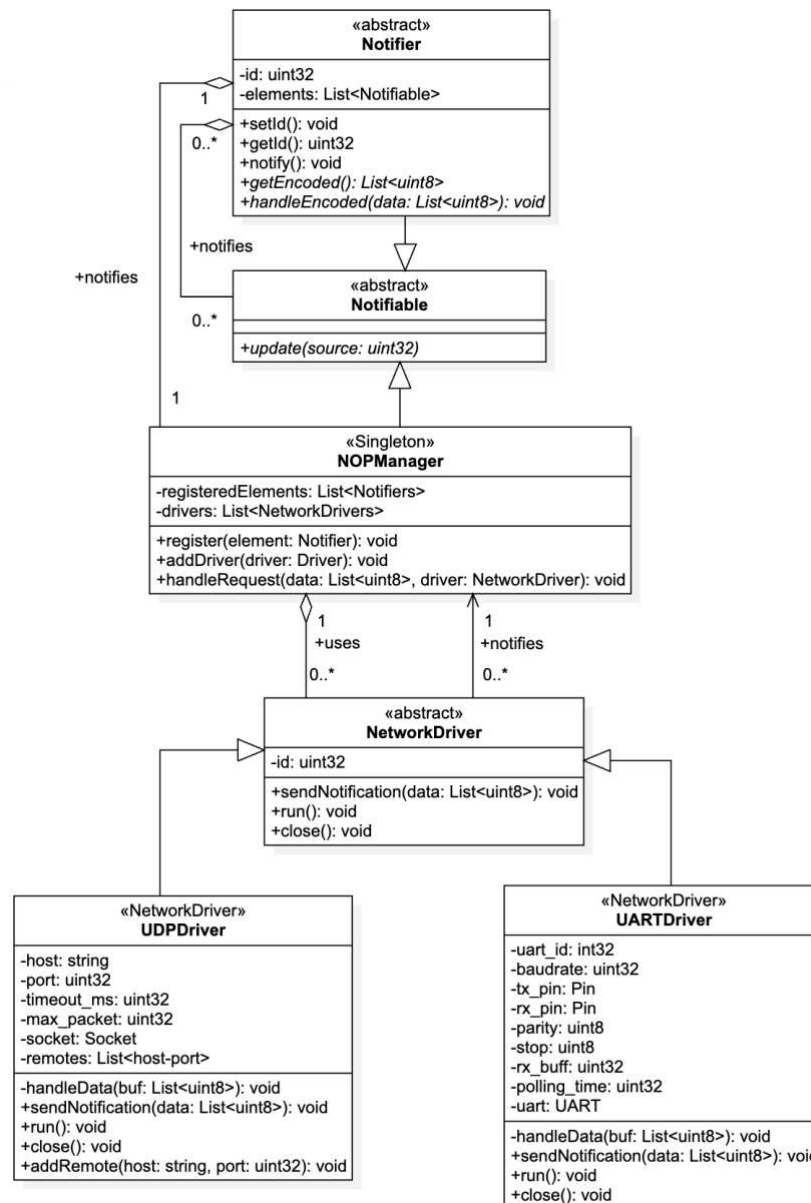
Figura 5 – Camadas do mecanismo de comunicação



Fonte: O autor.

Como parte do desenvolvimento do mecanismo de comunicação, foi elaborado o diagrama de classes para os componentes associados, conforme ilustrado na Figura 6. Entre os elementos criados, destaca-se a classe *NOPManager*, que atua como intermediária entre os componentes do *PON* e os *drivers* de comunicação.

Figura 6 – Diagrama de classes em UML do mecanismo de comunicação

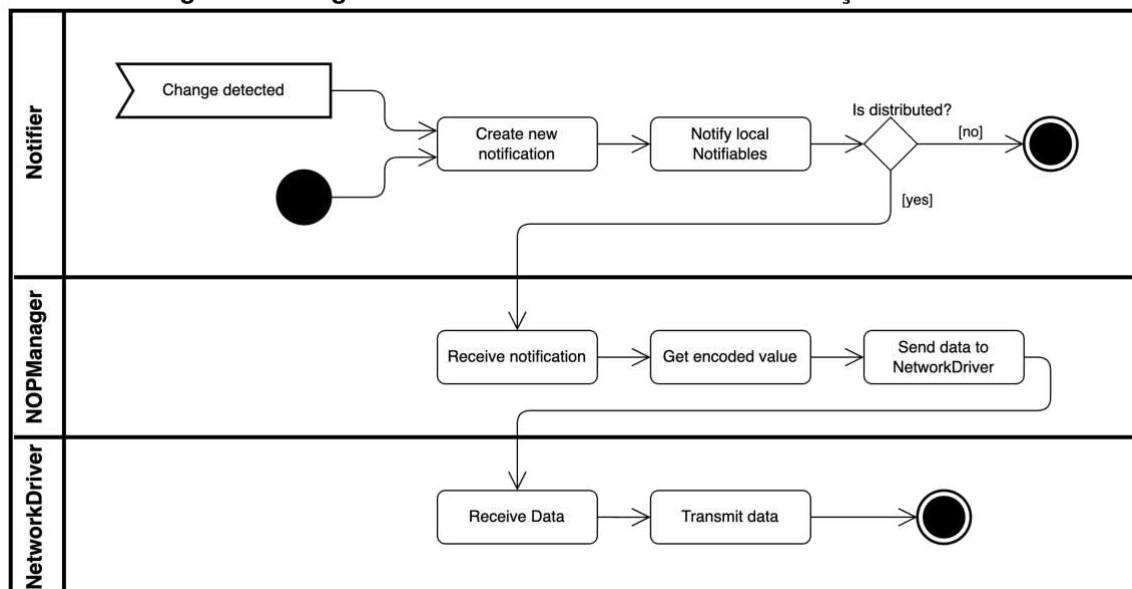


Fonte: O autor.

Para distinguir os componentes locais dos distribuídos, foi adotado um identificador único (*id*), representado por um inteiro de 32 bits. O valor zero foi reservado para indicar componentes locais, enquanto valores diferentes de zero identificam componentes distribuídos. Quando um componente recebe um *id* diferente de zero, ele automaticamente adiciona o NOPManager à sua lista de notificações, e NOPManager adiciona o elemento em um *HashMap* com seu *id*. Isto permite unificar o mecanismo de notificações para componentes locais e distribuídos, já que o NOPManager é visto como um *Notifiable* pelos outros elementos.

O funcionamento das três camadas, no contexto de elementos distribuídos, pode ser descrito por meio de dois fluxos principais. O primeiro fluxo, detalhado na Figura 7, corresponde ao processo iniciado no microcontrolador de origem da notificação. O fluxo tem início com a mudança de estado de um componente, que notifica suas dependências locais. Caso o componente seja distribuído, ele também notifica o *NOPManager*. Este, por sua vez, solicita o valor associado ao componente por meio do método *getEncoded*, que codifica a informação do elemento em formato JSON por padrão. Após a codificação, os dados são encaminhados aos *drivers* de comunicação ativos. Esses *drivers* têm a responsabilidade de transmitir as informações pelo meio físico definido, até os microcontroladores de destino, garantindo que os dados cheguem aos elementos relevantes no sistema distribuído.

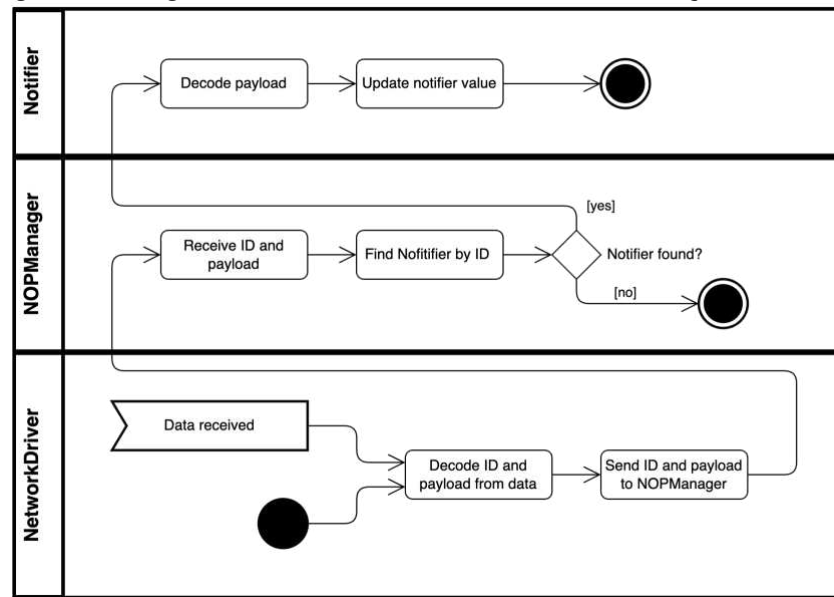
Figura 7 – Diagrama de atividades em UML de notificações locais.



Fonte: O autor.

O segundo fluxo, ilustrado na Figura 8, ocorre no microcontrolador remoto após o recebimento da notificação pelo driver de comunicação. Nesse caso, o processo segue uma lógica inversa em relação ao fluxo de origem. Ao receber os dados transmitidos, o componente correspondente no microcontrolador remoto utiliza a função *handleEncoded* para decodificar as informações recebidas. Em seguida, o componente processa a notificação, atualizando seu estado ou executando as ações correspondentes à notificação.

Figura 8 – Diagrama de atividades em UML das notificações remotas.



Fonte: O autor.

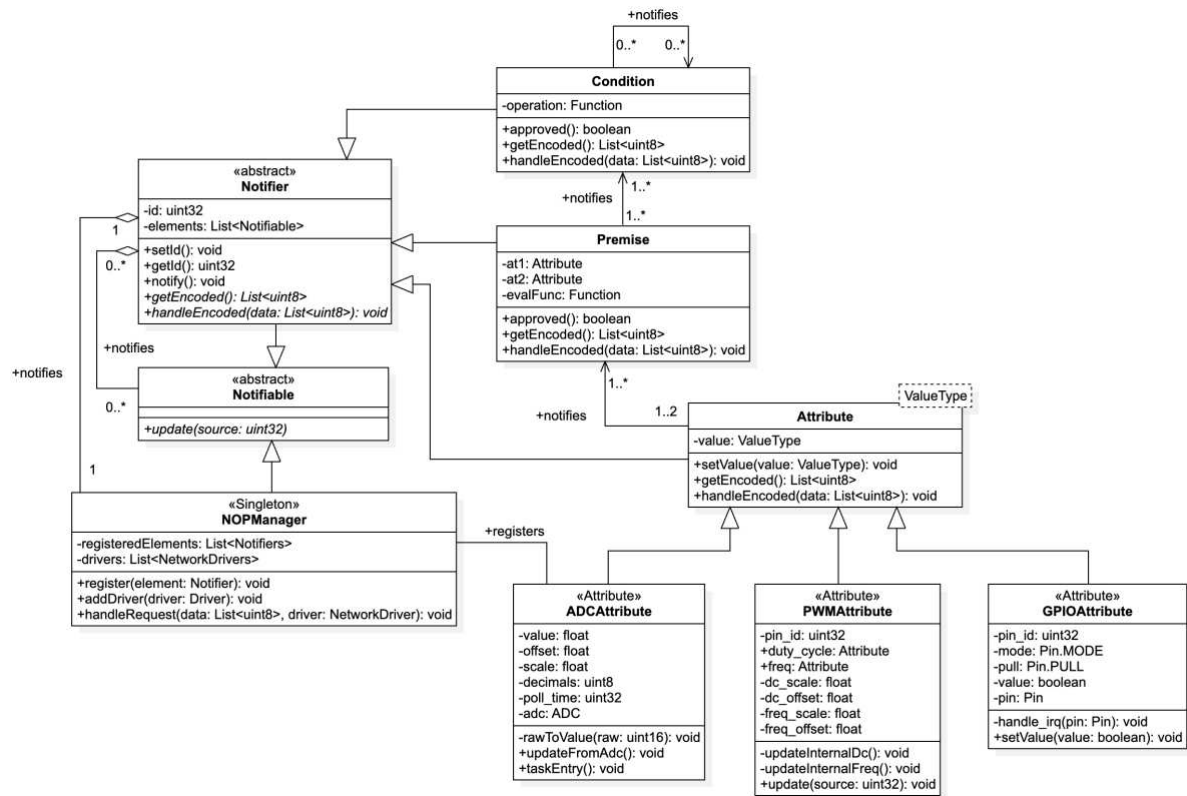
Esta abordagem garante a interoperabilidade entre notificações locais e distribuídas, simplificando mecanismo de notificações e a integração entre múltiplos microcontroladores, promovendo um fluxo de comunicação robusto, unificado e eficiente.

3.1.3 Projeto da integração com os periféricos do microcontrolador

Uma parte essencial do projeto do *framework* é a integração do mecanismo de notificação com os periféricos do microcontrolador, incluindo os pinos GPIO, os ADCs e os geradores de PWM. Essa abordagem elimina a necessidade de escrever código adicional para gerenciar as interações entre esses periféricos e o mecanismo de notificação, já que essas funcionalidades serão tratadas diretamente pelo *framework*.

Para o desenvolvimento desses componentes, foi inicialmente elaborado um diagrama de classes, representado na Figura 9. Decidiu-se definir os periféricos como especializações da classe *Attribute*. Essa estrutura mantém uma interface unificada com os demais componentes do *framework*, garantindo consistência, eficiência e simplicidade na integração.

Figura 9 – Diagrama de classes em UML da integração com os periféricos do microcontrolador.



Fonte: O autor.

O *GPIOAttribute* é o mais simples entre os três componentes integrados, pois essencialmente funciona como um *Attribute* com valor booleano associado a um pino digital, assumindo valores lógicos 0 (*false*) ou 1 (*true*). Sua criação exige apenas a definição do pino ao qual estará vinculado e do modo de funcionamento desejado. O modo de funcionamento pode ser configurado como entrada ou saída, modificando o comportamento do *GPIOAttribute*.

No Código 5, é apresentado um exemplo prático de utilização do componente *GPIOAttribute* em seus dois modos de operação. O exemplo consiste em um botão, configurado como entrada utilizando o *GPIOAttribute*, e um LED, configurado como saída através do mesmo componente, que são instanciados nas linhas 4 e 5. O comportamento implementado no código inverte o estado do LED sempre que o atributo associado ao pino onde está conectado o botão - *atBtn* - muda seu estado para "False", que é implementado no código da linha 8 até a linha 14.

Código 5 – Exemplo de uso do *GPIOAttribute*.

```

1 | btn_pin = 1 # Pino do botão
2 | led_pin = 15 # Pino do LED
3 |
4 | atBtn = GPIOAttribute(btn_pin, Pin.IN, pull=Pin.PULL_UP)
5 | atLed = GPIOAttribute(led_pin, Pin.OUT)
6 |
7 | # Muda estado do LED
8 | Rule(atBtn == False,
9 |     Action([
10 |         Instigation([
11 |             lambda : atLed.set_value(not atLed.value)
12 |         ])
13 |     ])
14 | )

```

Fonte: O autor.

A utilização do *GPIOAttribute* elimina a necessidade de funções adicionais para atualizar manualmente o valor de um atributo toda vez que o botão é acionado. Além disso, dispensa a criação de regras explícitas para sincronizar o estado do LED, pois o *GPIOAttribute* gerencia automaticamente essas interações. Esse recurso simplifica o desenvolvimento, reduz a complexidade do código e minimiza o risco de erros relacionados à atualização manual de estados ou sincronização entre componentes.

Quando o *GPIOAttribute* é utilizado no modo de entrada, seu valor não pode ser atribuído diretamente, pois ele estará vinculado ao estado do pino, que será atualizado por meio de uma interrupção. Essa interrupção é responsável por modificar o valor do *Attribute* de acordo com o estado do pino e por disparar as notificações associadas. A interrupção pode ser configurada para ser ativada em diferentes condições: a cada mudança no estado do pino ou exclusivamente em bordas de subida ou descida. No entanto, quando o *GPIOAttribute* é configurado no modo de saída ele atua como qualquer *Attribute* booleano, com a diferença que o seu valor também é representado no estado do pino associado.

Para a integração com o ADC do microcontrolador, foi projetado o *ADCAAttribute*, responsável por obter seu valor a partir da leitura de um pino analógico, com o objetivo de facilitar o uso de potenciômetros ou sensores analógicos. Para sua criação, é necessário apenas especificar o pino ao qual será associado. O valor do ADC é obtido por meio de *polling*, com o intervalo de leitura configurável no momento da construção. Além disso, considerando que é comum realizar deslocamentos e/ou ajustes de escala no valor bruto do ADC, foi projetada a possibilidade de configurar

esses parâmetros diretamente no *ADCAttribute*. Com isso, o valor lido é automaticamente ajustado, eliminando a necessidade de alterações adicionais para seu uso posterior.

Para integrar o gerador PWM do microcontrolador ao sistema, foi desenvolvido o *PWMAttribute*, um componente com comportamento diferenciado em relação a um *Attribute* convencional. Esse componente foi projetado para gerenciar dois parâmetros fundamentais do gerador PWM: a frequência e o ciclo de trabalho. Para isso, o *PWMAttribute* incorpora dois *Attributes* separados, um para armazenar cada parâmetro. Sempre que qualquer um desses *Attributes* é alterado, o *PWMAttribute* é notificado e, automaticamente, aplica as mudanças ao gerador PWM do microcontrolador. Esse mecanismo assegura que os parâmetros configurados sejam refletidos de forma precisa no sinal gerado, sem necessidade de intervenções manuais adicionais.

O Código 6 apresenta um exemplo prático e direto do uso integrado dos componentes *ADCAttribute* e *PWMAttribute*, demonstrando como a intensidade de um LED pode ser controlada por meio de um potenciômetro. Nesse exemplo, basta configurar os pinos associados a cada componente e, no caso do *ADCAttribute*, ajustar a escala para que seu valor esteja dentro dos limites válidos do ciclo de trabalho do gerador PWM. Esses limites e outros detalhes de cada componente serão detalhados na próxima seção.

Código 6 – Exemplo de uso do *ADCAttribute* e *PWMAttribute*.

```
1 | pot_pin = 28 # Pino do potenciômetro
2 | led_pin = 16 # Pino do led
3 |
4 | adc_scale = 1/(2**16) # Ajuste da escala do adc -> duty cycle
5 |
6 | atPot = ADCAttribute(pot_pin, scale=adc_scale)
7 |
8 | atPwmLed = PWMAttribute(led_pin, duty_cycle=atPot)
```

Fonte: O autor.

Essa abordagem simplifica significativamente a leitura de valor analógico e o controle de sinais PWM, permitindo que os desenvolvedores utilizem diretamente os componentes do PON para tarefas como controle de LEDs, motores, servomotores e outros dispositivos que dependem de sinais PWM.

3.2 Interface do *framework*

O *Framework* PON MicroPython 1.0 foi inteiramente desenvolvido na linguagem de programação MicroPython, utilizando apenas os pacotes nativos incluídos na própria linguagem, sem depender de bibliotecas externas. Para acessar os elementos deve-se importar o pacote principal como é apresentado no início do Código 7.

Código 7 – FBE da aplicação sensores implementada utilizando o *framework*.

```

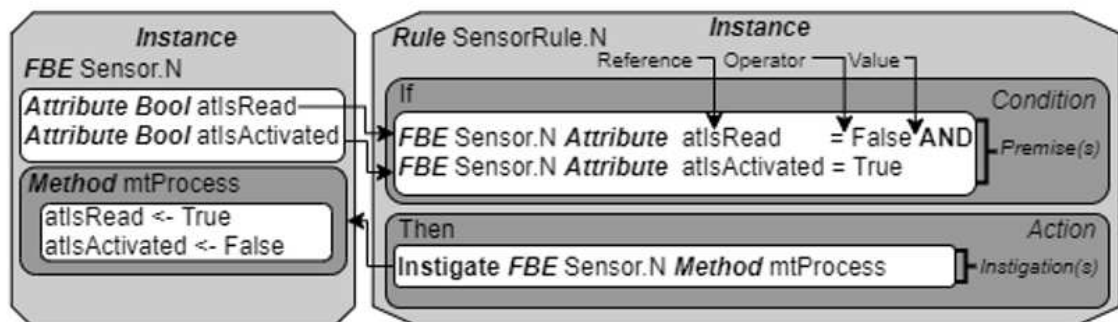
01 | from pon import *
02 |
03 | class Sensor:
04 |     def __init__(self):
05 |         self.atIsRead = Attribute(False)
06 |         self.atIsActivated = Attribute(False)
07 |
08 |         self.ruleSensor = Rule((self.atIsRead == False) & (self.atIsActivated == True),
09 |                                Action([
10 |                                    Instigation([
11 |                                        self.mtProcess
12 |                                    ])
13 |                                ]))
14 |
15 |
16 |     def mtProcess(self):
17 |         print('Processed')
18 |         self.atIsRead.value = True
19 |         self.atIsActivated.value = False

```

Fonte: O autor.

Adicionalmente, na Figura 10 é possível observar a definição de um *FBE*, enquanto o Código 7 apresenta sua implementação utilizando o *framework*. Este código ilustra a sintaxe final de todos os componentes do PON, evidenciando a clareza e simplicidade proporcionadas pelo *framework* no desenvolvimento de aplicações.

Figura 10 – FBE da aplicação de sensores.



Fonte: Adaptado de Neves (2021)

O exemplo anterior demonstra apenas o uso básico do *framework*, evidenciando sua funcionalidade essencial. No entanto, o *framework* foi projetado

para incluir uma variedade de componentes adicionais, cada um com diferentes opções de configuração para atender a diversos cenários de aplicação. A seguir, serão detalhados todos os componentes incorporados ao *framework*, destacando suas funcionalidades, opções de configuração e como podem ser utilizados.

3.2.1 *Fact-Base Element*

Os *FBEs* são implementados como classes comuns do Python, com o objetivo principal de encapsular os demais elementos do PON. Embora seu uso não seja estritamente necessário para a definição ou funcionamento dos outros componentes do *framework*, sua utilização é altamente recomendada. O uso de *FBEs* permite organizar os componentes de maneira mais clara e coesa, facilitando a compreensão e a manutenção das aplicações desenvolvidas. Essa abordagem também contribui para uma estrutura mais modular, permitindo uma melhor reutilização de código em projetos futuros. Ademais, isso respeita os princípios do PON desde sua origem, nos quais *FBEs* são fundamentais.

3.2.2 *Attribute*

O *framework* possui diferentes implementações do componente *Attribute*, já que além de sua versão simples (que somente armazena um valor), possui versões que estão integradas com os periféricos do microcontrolador.

Para criar um *Attribute* simples, basta utilizar o construtor da classe *Attribute*, fornecendo os parâmetros indicados no Quadro 3. É importante destacar que não é necessário especificar o tipo do valor que o atributo armazenará, pois este é automaticamente inferido no momento da sua criação, com base no valor inicial fornecido. Os tipos de dado aceito são valores booleanos, *strings*, números inteiros ou de ponto flutuante e outros tipos (classes) que sejam possíveis serializar no formato JSON. Adicionalmente, se o componente precisa ser distribuído, deve-se definir um valor para o seu *id*, que é do tipo inteiro de 32bits sem sinal, que deve ser único em toda a rede de distribuição.

Quadro 3 – Parâmetros de construção do *Attribute*.

Parâmetro	Requerido	Valor inicial	Descrição
value	Sim	-	Valor inicial do <i>Attribute</i> .
id	Não	0	Identificador do componente (uint32).

Fonte: O autor.

Para acessar ou modificar o valor de um *Attribute*, utiliza-se o elemento “.value”, como demonstrado no Código 8. Esse elemento está vinculado a uma função responsável por gerar notificações sempre que o valor for alterado. Além disso, caso seja necessário configurar um novo valor diretamente, pode-se empregar o método “setValue”, onde ambas formas garantem o disparo das notificações correspondentes, assegurando que todas as dependências sejam atualizadas conforme o esperado.

Código 8 – Leitura e escrita do valor do *Attribute*.

```

atSensorPos = Attribute(100)
# Leitura do valor
atSensorPos.value
# Escrita do valor
atSensorPos.value = 10
atSensorPos.setValue(10)
# Torna o Attribute distribuído
atSensorPos.setId(0x01);

```

Fonte: O autor.

Além de *Attribute* dito simples, o *framework* também inclui componentes específicos para a interação com periféricos do microcontrolador. O *GPIOAttribute* permite a leitura e escrita em pinos digitais, trabalhando com valores binários. Por sua vez, o *ADCAttribute* é projetado para entradas analógicas, sendo utilizado para capturar valores provenientes de sensores analógicos. Por fim, o *PWMAttribute* é responsável pelo controle de geradores PWM, possibilitando a configuração de frequência e *ciclo de trabalho do sinal*.

Para criar um *GPIOAttribute*, é necessário especificar o pino ao qual ele estará associado no momento de sua criação, utilizando o construtor da classe. Além disso, o *GPIOAttribute* oferece diversas configurações adicionais que podem alterar seu comportamento, permitindo adaptá-lo a diferentes casos de uso, essas configurações estão detalhadas no Quadro 4.

Quadro 4 – Parâmetros de construção do *GPIOAttribute*.

Parâmetro	Requerido	Valor inicial	Descrição
pin_id	Sim	-	Identificador do pino associado do microcontrolador.
mode	Não	IN	Modo de operação. (IN, OUT)
pull	Não	PULL_UP	Configura um resistor de <i>pull-up</i> ou de <i>pull-down</i> no pino. (Unicamente para mode=IN)
value	Não	-	Valor inicial do <i>Attribute</i> . (Unicamente para mode=OUT)
id	Não	0	Identificador do componente (uint32).

Fonte: O autor.

Quando há necessidade de realizar a leitura de um valor analógico, pode-se instanciar um *ADCAttribute*, que exige apenas a especificação do pino do ADC a ser utilizado. Além dessa configuração básica, o *ADCAttribute* oferece opções para ajustar o valor bruto lido, permitindo aplicar deslocamento ou mudanças de escala diretamente. Também é possível configurar a frequência de leitura do ADC, garantindo flexibilidade na adaptação a diferentes cenários. Essas opções estão detalhadas no Quadro 5.

Quadro 5 – Parâmetros de construção do *ADCAttribute*.

Parâmetro	Requerido	Valor inicial	Descrição
pin_id	Sim	-	Identificador do pino associado do microcontrolador.
offset	Não	0	Deslocamento do valor do <i>Attribute</i> .
scale	Não	1	Ajuste de escala do valor bruto do ADC.
poll_time	Não	1	Tempo de <i>polling</i> do ADC em milissegundos.
decimals	Não	0	Número de casas decimais do valor.
id	Não	0	Identificador do componente (uint32).

Fonte: O autor.

Para gerar um sinal PWM, o *PWMAttribute* pode ser utilizado, sendo necessário apenas especificar o pino do gerador PWM ao qual ele será associado. Este componente se diferencia dos demais, pois, para controlar tanto a frequência quanto o *duty cycle* do sinal, são criados dois *Attributes* distintos, um para cada parâmetro. Cada um desses *Attributes* possui seus respectivos parâmetros de ajuste, proporcionando flexibilidade para configurar o sinal PWM de acordo com as necessidades da aplicação. E esses parâmetros estão descritos no Quadro 6.

Quadro 6 – Parâmetros de construção do *PWMAttribute*.

Parâmetro	Requerido	Valor inicial	Descrição
pin_id	Sim	-	Identificador do pino associado do microcontrolador.
duty_cycle	Não	Attribute(0)	Attribute do <i>duty_cycle</i> do sinal gerado. Entre 0 e 1.
freq	Não	Attribute(1000)	Attribute da frequência do sinal gerado em Hertz.
dc_scale	Não	1	Ajuste de escala do valor do <i>duty_cycle</i> .
dc_offset	Não	0	Deslocamento do valor do <i>duty_cycle</i> .
freq_scale	Não	1	Ajuste de escala do valor da frequência.
freq_offset	Não	0	Deslocamento do valor da frequência.
id	Não	0	Identificador do componente (uint32).

Fonte: O autor.

As diversas implementações e integrações foram desenvolvidas com o objetivo de facilitar o uso dos *Attributes*, tornando o *framework* o mais transparente possível para o programador. A ideia foi minimizar a complexidade da interação com os *Attributes*, permitindo que o desenvolvedor se concentre na lógica da aplicação, enquanto o *framework* gerencia automaticamente os aspectos relacionados à comunicação e interação com os periféricos do microcontrolador.

3.2.3 *Premise*

Os objetos de *Premises* podem ser instanciados de forma simplificada utilizando os operadores lógicos (`==`, `!=`, `>`, `<`, `>=`, `<=`) entre duas instâncias de *Attributes* ou entre um *Attribute* e um valor, como apresentado no Código 9. Isso facilita a criação das *Premises*, pois elimina a necessidade de invocar diretamente o construtor, tornando o código mais conciso e legível. Além disso, por padrão, as *Premises* têm o seu *id* igual a zero. No entanto, esse *id* pode ser modificado utilizando a função “`setId`”, que ativa as funções distribuídas da *Premise* no caso em que é definido um *id* diferente de zero.

Código 9 – Exemplo de criação de *Premises*.

```

atSensorA = Attribute(100)
atSensorB = Attribute(100)

prEqual = atA == atB
prMinRequired = atA > 10
# Torna prEqual distribuída
prEqual.setId(0x20)

```

Fonte: O autor.

Quando uma *Premise* precisa ser acessada em outro microcontrolador, deve-se utilizar a *RemotePremise*, atribuindo-lhe o mesmo *id* que foi utilizado na *Premise* original. Dessa forma, o *framework* garante que as notificações sejam recebidas sempre que o estado da *Premise* original for alterado. Os parâmetros adicionais do construtor da *RemotePremise* estão descritos no Quadro 7, como a opção de definir um valor inicial.

Quadro 7 – Parâmetros de construção de *RemotePremise*.

Parâmetro	Requerido	Valor inicial	Descrição
id	Sim	-	Identificador do componente (uint32).
value	Não	False	Valor padrão da <i>Premise</i> até receber a notificação.

Fonte: O autor.

O Código 10 apresenta um pequeno exemplo de como acessar e utilizar a *Premise* “prEqual” declarada no exemplo do Código 9.

Código 10 – Exemplo de uso de *RemotePremise*.

```

prEnabled = atLogActive == True

# Relaciona valor a prEqual
prEqual = RemotePremise(0x20)

r1 = Rule(prEnabled & prEqual,
          Instigation([
            Action([
              lambda : print('Active and Equal')
            ])
          ]))

```

Fonte: O autor.

Essa interface facilita a criação de *Premises*, tanto nos casos em que elas devem ser compartilhadas por várias *Conditions*, quanto nas situações em que são usadas de forma isolada. Ela permite que as *Premises* sejam facilmente

representadas por uma variável, com uma sintaxe de criação simples e intuitiva, tornando o processo mais fácil para o desenvolvedor.

3.2.4 Condition

A criação de *Conditions* é simples e direta, como pode ser visto no exemplo do Código 11. As operações lógicas de conjunção e disjunção são implementadas pelos operadores “&” e “|”, respectivamente. Note-se que esses operadores funcionam exclusivamente entre instâncias de *Premise*, sejam elas representadas por variáveis (como no Código 10) quanto criadas no momento da criação da *Condition* (Código 11).

Código 11 – Exemplo de criação de uma *Condition*.

```
atIsRead = Attribute(False)
atIsActivated = Attribute(False)
# Cria a Condition
cnSensor = (atIsRead == False) & (atIsActivated == True)
# Ativa distribuição da Condition
cnSensor.setId(0xA0); # id = 0xA0 = 160
rlSensor = Rule(cnRule,
                Action([
                    Instigation([
                        lambda : print('Processed'),
                        atIsRead.setValue(True)
                    ])
                ]))
```

Fonte: O autor.

Da mesma forma que para a *Premise*, quando uma *Condition* precisa ser acessada em outro microcontrolador, deve-se utilizar a *RemoteCondition*, atribuindo-lhe o mesmo *id* que foi utilizado na *Condition* original. Dessa forma, o *Framework* PON MicroPython 1.0 garante que as notificações sejam recebidas sempre que o estado da *Condition* original for alterado. Os parâmetros adicionais da *RemoteCondition* estão descritos no Quadro 8, como a opção de definir um valor inicial.

Quadro 8 – Parâmetros de construção de *RemoteCondition*.

Parâmetro	Requerido	Valor inicial	Descrição
id	Sim	-	Identificador do componente (uint32).
value	Não	-	Valor padrão da <i>Condition</i> até receber a notificação.

Fonte: O autor.

O Código 12 apresenta um pequeno exemplo de como acessar e utilizar a *Condition* “cnSensor” declarada no exemplo do Código 11.

Código 12 – Exemplo de uso de *RemoteCondition*

```
# Relaciona valor a cnSensor
cnSensor = RemoteCondition(0xA0)
rlSensor = Rule(cnSensor,
    Action([
        Instigation([
            lambda : print('Processed'),
            atIsRead.setValue(True)
        ])
    ])
)
```

Fonte: O autor.

Seguindo o padrão dos demais elementos do *framework*, a criação e uso das *Conditions* é simples e direta, utilizando no máximo a sobrecarga de operadores para minimizar a quantidade de trabalho do desenvolvedor. Isso permite que as *Conditions* sejam definidas de forma rápida e intuitiva.

3.2.5 *Method*

Para o *Method*, não foi desenvolvido um novo elemento, pois foram utilizados os próprios métodos do MicroPython. A criação de um componente específico para isso não traria funcionalidades adicionais, mas adicionaria uma carga desnecessária ao *framework*. Dessa forma, optou-se por aproveitar a flexibilidade dos métodos padrão da linguagem, mantendo o *framework* leve e eficiente. Exemplos de criação de métodos normais e *lambda* podem ser observados no Código 13.

Código 13 – Exemplo de criação de *Methods*.

```
mtA = lambda : print ("Executed A")

def mtB() :
    print("Executed B")
```

Fonte: O autor.

Ao utilizar os métodos nativos do MicroPython para representar os *Methods*, o *framework* oferece flexibilidade para que o desenvolvedor aproveite as funcionalidades da linguagem sem a necessidade de elementos adicionais.

3.2.6 *Instigation*

As *Instigations* são responsáveis por executar seus métodos em sequência quando são ativadas. Sua construção é simples, pois envolve apenas fornecer uma lista contendo todos os *Methods* que devem ser executados no momento de criação, como demonstrado no exemplo do Código 14.

Código 14 – Exemplo de criação de uma *Instigation*.

```
mtA = lambda : print ("Executed A")

def mtB() :
    print("Executed B" )

insA = Instigation([mtA, mtB, lambda : print("Executed C")])
```

Fonte: O autor.

Dessa forma, as *Instigations* oferecem uma maneira prática e eficiente de agrupar e executar múltiplos *Methods* em uma ordem específica. Isso garante flexibilidade e organização, tornando o processo de desenvolvimento mais organizado.

3.2.7 *Action*

As *Actions* são responsáveis por executar as *Instigations* associadas a elas. Em seu construtor, elas recebem uma lista das *Instigations* que devem ser executadas quando acionadas. Isso permite que múltiplas sequências de métodos sejam agrupadas e executadas de maneira controlada, de acordo com a necessidade da aplicação. Um exemplo da criação de uma *Action* pode ser visto no Código 15.

Código 15 – Exemplo de criação de uma *Action*.

```
mtA = lambda : print ("Executed A")
mtB = lambda : print ("Executed B")

insA = Instigation([mtA, mtB, lambda : print("Executed C")])
insB = Instigation([lambda : print("Executed D")])

acA = Action([insA, insB])
```

Fonte: O autor.

Assim, as *Actions* proporcionam uma forma estruturada de orquestrar a execução de várias *Instigations* de maneira controlada, permitindo que ações

complexas sejam realizadas com um único comando. Essa abordagem simplifica a lógica da aplicação, tornando o código mais modular e fácil de entender.

3.2.8 Rule

A *Rule* é o componente central do *framework*, pois é responsável por unificar toda a parte lógica para com a execução das ações, *i.e.*, processamento lógico-causal. Em seu construtor, ela recebe apenas uma *Condition* e uma *Action*, como pode ser observado no Código 16. Quando a *Condition* é satisfeita, a *Action* correspondente é executada.

Código 16 – Exemplo de criação de uma *Rule*.

```
atA = Attribute (10)
atB = Attribute (20)
atC = Attribute (30)

prA = atB > 10
prB = atA != atC

cnA = prA & prB

mtA = lambda : print("Executed A")
mtB = lambda : print("Executed B")

insA = Instigation ([mtA, mtB])

acA = Action ([insA])

r1A = Rule(cnA, acA)
```

Fonte: O autor.

Adicionalmente, caso não seja necessário o compartilhamento de uma *Premise*, *Condition*, *Action* ou *Instigation*, o *framework* permite a definição direta desses componentes. Essa abordagem simplifica ainda mais a criação de expressões lógicas, pois elimina a necessidade de variáveis intermediárias, permitindo que os componentes sejam definidos e utilizados localmente dentro da aplicação. O exemplo apresentado no Código 17 ilustra como essa definição direta permite resumir a declaração da *Rule* do exemplo do Código 16.

Código 17 – Exemplo de criação de uma *Rule* sem compartilhamento.

```

atA = Attribute(10)
atB = Attribute(20)
atC = Attribute(30)

r1A = Rule((atB > 10) & (atA != atC),
           Action([
               Instigation([
                   lambda : print("Executed A"),
                   lambda : print("Executed B")
               ])
           ])
)

```

Fonte: O autor.

Essa abordagem facilita a implementação de soluções mais diretas e locais, fornecendo ao desenvolvedor mais controle sobre a forma de organizar as definições dos componentes. Assim, o *framework* mantém sua adaptabilidade, atendendo tanto a cenários simples quanto a situações mais complexas.

3.2.9 *Drivers* de comunicação

Neste trabalho, foram implementados dois *drivers* de comunicação. O primeiro, *UARTDriver*, é responsável por transmitir as notificações pela interface UART do microcontrolador. O segundo, *UDPDriver*, permite a transmissão de notificações através da rede Ethernet, utilizando o protocolo UDP sobre um sistema de comunicação.

Os *drivers* de comunicação são responsáveis por fornecer a funcionalidade de distribuição ao *framework*, pois são encarregados de transportar as notificações através de um meio de transmissão específico. Eles foram projetados de forma modular e independente do meio pelo qual a notificação é transmitida, permitindo flexibilidade e facilitando a implementação de novos *drivers* conforme necessário. Essa abordagem garante que o *framework* possa ser facilmente adaptado para diferentes tipos de comunicação.

O *UARTDriver* foi implementado para ser de fácil utilização, com todos os seus parâmetros de construção definidos com valores predefinidos. Essa abordagem visa simplificar sua aplicação em cenários comuns, permitindo que o desenvolvedor o utilize sem a necessidade de ajustes adicionais na maioria dos casos. No entanto, para situações em que sejam necessárias modificações específicas na configuração da interface UART, o *driver* oferece uma ampla gama de opções em seu construtor.

Os parâmetros disponíveis para configuração detalhada podem ser consultados no Quadro 9.

Quadro 9 – Parâmetros de construção do *UARTDriver*

Parâmetro	Requerido	Valor inicial	Descrição
uart	Não	1	Identificador do UART do microcontrolador a ser utilizado.
baudrate	Não	57600	Taxa de transmissão utilizada na comunicação UART.
tx_pin	Não	Pin(4)	Pino de transmissão.
rx_pin	Não	Pin(5)	Pino de recepção.
parity	Não	None	Paridade a ser utilizada. (<i>None, Odd, Even</i>)
stop	Não	1	<i>Stop bit</i> a ser utilizado, pode ser 0 ou 1.
rx_buf	Não	1024	Tamanho do <i>buffer</i> de leitura em bytes.
polling_time	Não	10	Tempo de <i>polling</i> em milissegundos.
verbose	Não	False	Ativa mensagens de <i>debug</i> .

Fonte: O autor.

O *UDPDriver* segue o mesmo padrão de simplicidade do *UARTDriver*, não exigindo parâmetros obrigatórios no momento de sua construção. Contudo, também oferece uma ampla gama de configurações, como ilustrado no Quadro 10.

Para seu correto funcionamento, é imprescindível que o microcontrolador esteja conectado a uma rede por meio de uma interface Ethernet ou Wi-Fi, que será utilizada para o envio das notificações. Além disso, é necessário adicionar manualmente, por meio do método “*addRemote*”, o endereço IP e a porta do servidor UDP para cada microcontrolador remoto.

Quadro 10 – Parâmetros de construção do *UDPDriver*.

Parâmetro	Requerido	Valor inicial	Descrição
host	Não	0.0.0.0	IP do servidor UDP.
port	Não	57600	Porta do servidor UDP.
timeout_ms	Não	1	Tempo de <i>polling</i> em milissegundos.
max_packet	Não	1024	Tamanho máximo do pacote em bytes.
verbose	Não	False	Ativa mensagens de <i>debug</i> .

Fonte: O autor.

Para integrar os *drivers* de comunicação ao *framework*, eles devem ser adicionados à cadeia de notificações por meio do *NOPManager*, que atua como intermediário, redirecionando as notificações para os *drivers* configurados. No Código

18, é apresentado um exemplo do processo de criação e associação dos *drivers* ao *NOPManager*. Isso garante que qualquer notificação de um componente distribuído seja comunicada aos demais microcontroladores associados.

Código 18 – Exemplo de uso de múltiplos *drivers* de comunicação.

```
# Cria drivers
driverUDP1 = UDPDriver()
driverUART1 = UARTDriver()

# Registra microcontroladores remotos
driverUDP1.addRemote('informer-25.local', 5050)
driverUDP1.addRemote('informer-10.local', 5050)

# Adiciona drivers ao PONManager
NOPManager.GetInstance().addDriver(driverUDP1)
NOPManager.GetInstance().addDriver(driverUART1)
```

Fonte: O autor.

Esta abordagem da implementação dos *drivers* de comunicação no *framework* garante a flexibilidade e a escalabilidade necessárias para distribuir notificações em sistemas com múltiplos microcontroladores. Tanto o *UARTDriver* quanto o *UDPDriver* ilustram como diferentes meios de comunicação podem ser integrados de forma modular e configurável, atendendo às demandas específicas de cada aplicação.

3.3 Projeto das aplicações de teste

Esta seção apresenta o projeto de aplicações que demonstram o uso das funcionalidades do *framework* proposto. Esses exemplos foram projetados com o objetivo de validar os componentes desenvolvidos, ilustrar sua integração em diferentes contextos de uso e compará-lo frente a implementações sem o uso do *framework*.

3.3.1 Aplicação de sensores

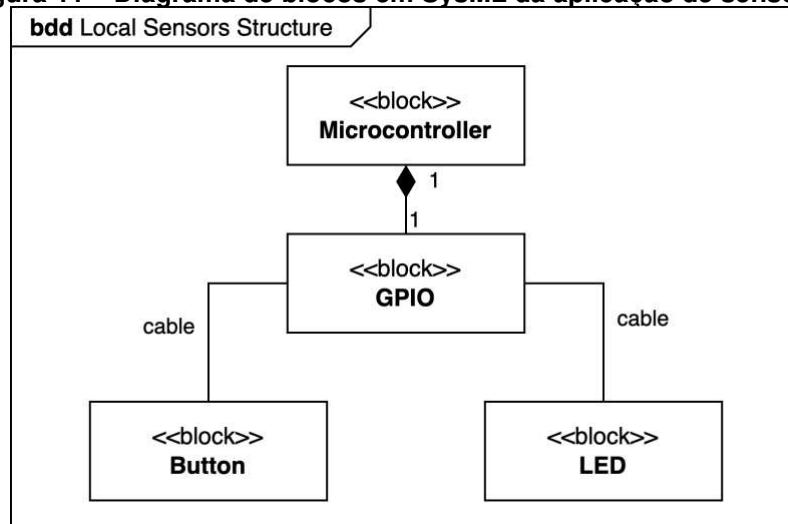
A aplicação de sensores foi desenvolvida com base no modelo *FBE* apresentado na Figura 2, amplamente utilizado para avaliar *frameworks* do PON devido à sua simplicidade, que permite medir com facilidade e precisão o custo adicional introduzido pelo mecanismo de notificação. A aplicação é composta por um sensor que possui um estado que pode ser “ativado” ou “desativado” e um indicador para identificar se o estado foi processado. Cada vez que o sensor é ativado e não

processado ele executa um método que avisa ao usuário da ativação (geralmente por um texto), e marca o estado como processado.

Para verificar o desempenho do *framework*, foram idealizadas duas implementações: uma com componentes totalmente locais e outra distribuída. Essa abordagem permite analisar de forma independente os custos associados ao mecanismo de notificação interna e ao mecanismo de transmissão entre dispositivos. Além disso, para ambas as implementações que utilizam o *framework*, foi projetada uma versão equivalente da aplicação sem o uso do *framework*, servindo como base comparativa.

No caso da aplicação totalmente local, o projeto foi baseado no diagrama de blocos apresentado na Figura 11. Esse diagrama inclui um microcontrolador, um botão e um LED. O botão tem a função de ativar o sensor, enquanto o LED funciona como um indicador visual, acendendo sempre que o sensor estiver em estado ativo.

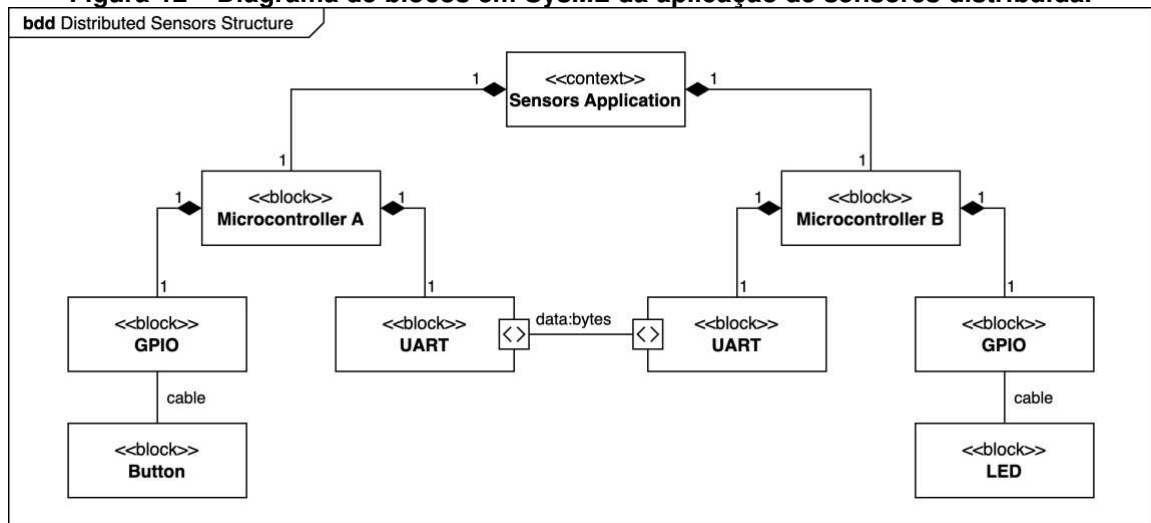
Figura 11 – Diagrama de blocos em SysML da aplicação de sensores.



Fonte: O autor.

Para validar as opções de distribuição do *framework*, foi projetada uma versão da aplicação que opera em dois microcontroladores distintos, conforme ilustrado na Figura 12. O funcionamento dessa versão é essencialmente idêntico ao da versão local, com a diferença de que o LED e o botão estão conectados a microcontroladores diferentes. As notificações entre os dispositivos são transmitidas por meio de uma conexão UART, permitindo avaliar o desempenho do *framework* em um ambiente distribuído.

Figura 12 – Diagrama de blocos em SysML da aplicação de sensores distribuída.



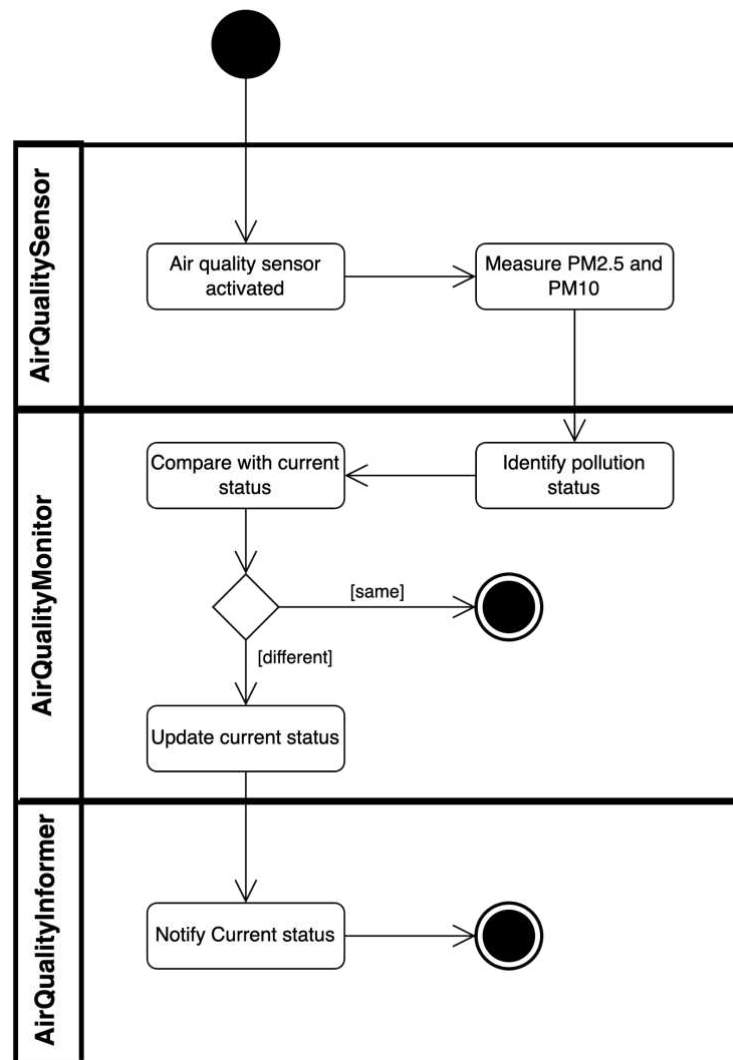
Fonte: O autor.

3.3.2 Aplicação de monitor de qualidade do ar

A aplicação *Air Quality Monitor* (AQM), conforme definida por Babu (2022), foi empregada para validar a facilidade proporcionada pelo novo *framework* na distribuição de aplicações. Esta aplicação monitora a qualidade do ar em um determinado momento, avaliando os níveis de partículas suspensas no ar. Há dois indicadores, PM2,5 e PM10, que correspondem à quantidade de partículas com diâmetro máximo de 2,5 e 10 micrômetros. Nos testes, foram avaliados os indicadores PM2,5 e PM10, e quanto menores seus valores, melhor a qualidade do ar naquele momento.

Essa aplicação possui três elementos principais, ilustrados na Figura 13. O *AirQualitySensor* é responsável por realizar as medições dos parâmetros de qualidade do ar, especificamente os níveis de PM2,5 e PM10. O *AirQualityMonitor* atua como o componente de processamento, encarregado de analisar os valores fornecidos pelos sensores e determinar o estado geral da qualidade do ar. Por fim, o *AirQualityInformer* é o elemento responsável por comunicar a qualidade do ar de cada parâmetro, de forma que sejam interpretáveis para os usuários.

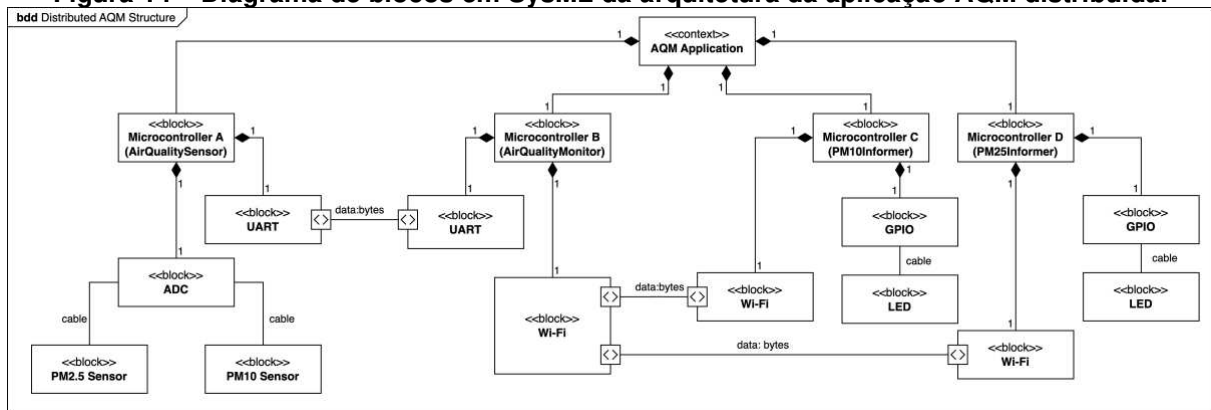
Figura 13 – Diagrama de atividades em SysML da aplicação de monitor de qualidade de ar.



Fonte: Adaptado de Babu (2022).

A partir dos elementos da aplicação, foram definidos *FBEs* individuais para cada componente, os quais foram distribuídos em quatro microcontroladores, conforme ilustrado no diagrama da Figura 14. O *AirQualitySensor* utiliza o *ADCAtribute* para realizar a leitura dos dois sensores (PM2,5 e PM10) e transmite as notificações para o monitor por meio do *UARTDriver*. Esta distribuição em quatro microcontroladores possui apenas objetivos de avaliação, mas em situações reais, poderia haver múltiplos locais de sensoriamento, e múltiplos locais de monitoramento dos valores, distribuídos por uma fábrica, por exemplo, onde seria possível monitorar as condições medidas nos diversos locais num só microcontrolador centralizado, ou mesmo monitorar visualmente a situação de todos os locais em cada microcontrolador.

Figura 14 – Diagrama de blocos em SysML da arquitetura da aplicação AQM distribuída.



Fonte: O autor.

O *AirQualitySensor* (microcontrolador A) é responsável por obter os dados dos sensores por meio de um bloco ADC, e então notificar os atributos distribuídos, em caso de mudanças nas leituras. O *AirQualityMonitor* (microcontrolador B), é responsável por receber os valores enviados pelos sensores e processá-los. Em seguida, transmite as notificações relacionadas ao estado da qualidade do ar para os *AirQualityInformers* (microcontroladores C e D) por meio do *UDPDriver*.

Por fim, existem dois *AirQualityInformers*, um para cada tipo de sensor. Esses componentes utilizam LEDs de cores diferentes para comunicar visualmente a qualidade do ar, permitindo uma identificação clara e rápida dos parâmetros monitorados.

3.4 Considerações

O Quadro 11 apresenta as propriedades elementares implementadas por cada *framework* analisado, incluindo o *Framework* PON MicroPython 1.0. Observa-se que o *framework* desenvolvido se destaca por implementar quase todas as propriedades elementares definidas, com a única exceção sendo o paralelismo via desacoplamento.

Quadro 11 – Propriedades elementares contempladas nas materializações do PON.

Framework	Programação em alto nível	Paralelismo via desacoplamento	Distribuição via desacoplamento	Código não redundante
FW PON CPP 2.0	Parcialmente via wizard	-	-	parcialmente
FW PON CPP 3.0	parcialmente	parcialmente	-	-
FW PON CPP 4.0	sim	sim	-	sim
FW PON CPP 4.5	sim	sim	-	sim
FW PON CPP 4.0 IoT	sim	sim	sim	sim
FW PON Java/C#	parcialmente	-	-	parcialmente
FW PON Erlang/Elixir	parcialmente	sim	suportado, porém não validado	-
FW PON Akka.Net	Parcialmente	sim	suportado, porém não validado	-
FW PON C# IoT	parcialmente	sim	sim	-
FW MicroPython 1.0	sim	-	sim	sim

Fonte: Adaptado de FIGUEIREDO (2022).

O Quadro 12 apresenta um resumo das principais contribuições para o desenvolvimento de tecnologia no contexto de sistemas distribuídos, incluindo a adição do *Framework* PON MicroPython 1.0. É importante destacar que as colunas relacionadas aos protocolos de transporte e de aplicação referem-se exclusivamente ao *UDPDriver*, uma vez que a estrutura modular do mecanismo de comunicação do *framework* permite a implementação de qualquer protocolo de transporte e/ou aplicação conforme necessários. Adicionalmente, essa modularidade possibilita o uso de outros meios de comunicação, como o *UARTDriver*, ampliando a flexibilidade e a aplicabilidade do *framework*.

Quadro 12 – Trabalhos realizados em PON no contexto de sistemas distribuídos.

Framework	Distribuição	Paralelismo	Entidades distribuídas	Protocolo de transporte	Protocolo de aplicação
FW PON CPP 2.0 + PONIP	Sim	Não	Attributes Premises	UDP/TCP	PONIP
FW PON CPP 3.0 + (LobeNOI + PONIP)	Sim	Sim	Attributes Premises	UDP/TCP	PONIP
FW PON Erlang/Elixir	Potencialmente	Sim	N/A	N/A	N/A
FW PON Akka.Net	Potencialmente	Sim	N/A	N/A	N/A
FW PON C# IoT	Sim	Sim	Atributes	UDP/TCP	HTTP Próprio
FW PON Java + Attributes Distribuídos	Sim	Não	Attributes	TCP	Próprio
MicroPON em IoT	Sim	Não	Atributes	TCP	Próprio
FW PON CPP 4.0 IoT	Sim	Sim	Attributes, Premises, Conditions, Rules, Actions, Instigations	TCP	MQTT
FW MicroPython 1.0 (UDPDriver)	Sim	-	Attributes, Premises, Conditions	UDP	Próprio

Fonte: Adaptado de FIGUEIREDO (2022).

Portanto, o *Framework* PON MicroPython 1.0 não apenas atende grande parte das propriedades fundamentais do PON, mas também se destaca como uma solução altamente adaptável. Sua capacidade de integração com diferentes protocolos de transporte e aplicação, aliada ao suporte a diversos meios de comunicação, como o *UARTDriver*, oferece uma maior flexibilidade, que é ideal para aplicações em sistemas embarcados que apresentam restrições de hardware ou exigências específicas de comunicação. Além disso, o *framework* facilita a interação direta com os periféricos do microcontrolador, permitindo o controle eficiente de sinais PWM, a leitura de ADCs e o gerenciamento de GPIOs de maneira simples e automatizada.

4 TESTES E RESULTADOS

Esta seção apresenta os testes do *framework* proposto, *Framework* PON MicroPython 1.0, com foco em sua funcionalidade e desempenho em diferentes cenários de aplicação. O objetivo é validar a eficácia do *framework*, explorando métricas de desempenho como latência, uso de memória, integração com periféricos e facilidade de uso.

4.1 Aplicação de sensores

A implementação da aplicação sensores foi fundamental para analisar o impacto das abstrações e funcionalidades do *framework* no desempenho e no desenvolvimento, tanto em aplicações locais quanto distribuídas. A comparação entre as quatro versões implementadas – local e distribuída, com e sem o uso do *framework* – permitiu observar suas vantagens e desvantagens em diferentes cenários. Todas essas versões encontram-se no repositório da aplicação¹.

Na versão totalmente local implementada com o *Framework* PON MicroPython 1.0, vista no Código 19, o uso do *GPIOAttribute* simplificou consideravelmente o desenvolvimento, eliminando a necessidade de configurar manualmente os pinos GPIO e suas interrupções, como foi exigido na versão orientada a objetos, que é apresentada no Código 20. Essa abstração reduziu a complexidade e o tempo de implementação, proporcionando uma interface mais intuitiva para o desenvolvedor.

¹ https://nop.dainf.ct.utfpr.edu.br/nop-implementations/frameworks/nop-framework-micropython-1_0/nop-framework-micropython-1_0-applications/sensors

Código 19 – FBE principal da aplicação *Sensors* local no PON.

```

class Sensor:
    def __init__(self):
        self.atIsRead = Attribute(False)
        self.atIsActivated = GPIOAttribute(18, Pin.OUT, value=False)
        self.ruleSensor = Rule((self.atIsRead == False) & (self.atIsActivated == True),
                                Action([
                                    Instigation([
                                        self.mtProcess
                                    ])
                                ])
                                )

    def mtProcess(self):
        print('Processed')
        self.atIsRead.value = True
        self.atIsActivated.value = False

async def main():
    sensor = Sensor()
    btn = GPIOAttribute(15, Pin.IN, pull=Pin.PULL_UP)
    pressed = Attribute(False)
    Rule((sensor.atIsActivated == False) & (btn == False) & (pressed == False),
        Action([Instigation([
            lambda : sensor.atIsActivated.set_value(True),
            lambda : sensor.atIsRead.set_value(False),
            lambda : pressed.set_value(True)]
        ])
        ])
    Rule((btn == True) & (pressed == True),
        Action([Instigation([lambda : pressed.set_value(False)]
        ])
    ])
    await NOPManager.GetInstance().run()

```

Fonte: O autor.

No Código 20 é apresentada a versão implementada no paradigma orientado a objetos.

Código 20 – Código da aplicação *Sensors* local no POO.

```

import asyncio as asyncio
from machine import Pin

class Sensor:
    def __init__(self):
        self.__read = False
        self.__activated = True
        self.__led = Pin(18, mode=Pin.OUT)
        self.update()

    def update(self):
        process = self.shouldProcess()
        self.__led.value(process)
        if process:
            self.mtProcess()

    def clearRead(self):
        self.__read = False

    def activate(self):
        self.__activated = True

    def shouldProcess(self):
        return (not self.__read) and (self.__activated)

    def mtProcess(self):
        print('Processed')
        self.__read = True
        self.__activated = False

async def main():
    sensor = Sensor()

    btn = Pin(15, mode=Pin.IN, pull=Pin.PULL_UP)

    def handle():
        sensor.clearRead()
        sensor.activate()

    btn.irq(lambda x: handle(), Pin.IRQ_FALLING)

    sensor.update()

    while True:
        await asyncio.sleep(0.001)
        sensor.update()

```

Fonte: O autor.

A versão distribuída destacou ainda mais a modularidade do *Framework* PON MicroPython 1.0, cuja implementação é apresentada no Código 21. Com alterações mínimas na versão local e como o uso do *UARTDriver*, foi possível implementar a distribuição das notificações entre os microcontroladores.

Código 21 – FBE principal da aplicação *Sensors* distribuída em PON.

```

class Sensor:
    def __init__(self):
        self.atIsRead = Attribute(False, 0x1001)
        self.atIsActivated = Attribute(False, id=0x1002)

        self.ruleSensor = Rule((self.atIsRead == False) & (self.atIsActivated == True),
                                Action([
                                    Instigation([
                                        self.mtProcess
                                    ])
                                ])
                                )

    def mtProcess(self):
        print('Processed')
        self.atIsRead.value = True
        self.atIsActivated.value = False

async def main():
    sensor = Sensor()
    btn = GPIOAttribute(15, Pin.IN, pull=Pin.PULL_UP)
    pressed = Attribute(False)
    Rule((sensor.atIsActivated == False) & (btn == False) & (pressed == False),
        Action([Instigation([
            lambda : sensor.atIsActivated.set_value(True),
            lambda : sensor.atIsRead.set_value(False),
            lambda : pressed.set_value(True)
        ])]))

    Rule((btn == True) & (pressed == True),
        Action([Instigation([lambda : pressed.set_value(False)
        ])]))

    driver = UARTDriver(verbose=False)
    NOPManager.GetInstance().addDriver(driver)

    await NOPManager.GetInstance().run()

```

Fonte: O autor.

Já na versão orientada a objetos, apresentada no Código 22, foi necessário desenvolver do zero um módulo de comunicação UART com suporte a *callbacks*, adicionando complexidade e aumentando o volume de código.

Código 22 – Parte do código da aplicação de sensores em POO².

```

| # ...
075 | class Sensor(UARTObserver):
076 |     def __init__(self, uartman: UARTManager):
077 |         self.__read = False
078 |         self.__activated = True
079 |         self.__led = Pin(18, mode=Pin.OUT)
080 |         self.__uartman = uartman
081 |
082 |         uartman.addObserver(self)
083 |
084 |         self.update()
| # ...
117 | async def main():
118 |     man = UARTManager()
119 |     sensor = Sensor(man)
120 |
121 |     btn = Pin(15, mode=Pin.IN, pull=Pin.PULL_UP)
122 |
123 |     def handle():
124 |         print("Pressed")
125 |         sensor.clearRead()
126 |         sensor.activate()
127 |
128 |     asyncio.create_task(man.run())
129 |
130 |     btn.irq(lambda x: handle(), Pin.IRQ_FALLING)
131 |
132 |     while True:
133 |         await asyncio.sleep(0.001)
134 |         sensor.update()

```

Fonte: O autor.

Para a medição da latência entre o acionamento do botão e a ativação do LED, foi utilizado um microcontrolador externo dedicado à medição. Esse microcontrolador foi conectado diretamente tanto ao botão quanto ao LED e configurado para registrar o tempo entre a detecção do acionamento do botão e a ativação do LED, utilizando interrupções para garantir precisão. Essa metodologia permitiu isolar o impacto dos processos internos de cada versão implementada, tanto local quanto distribuída, assegurando a coleta de dados consistentes sobre os tempos de resposta.

Foram realizadas dez medições para cada implementação, com o objetivo de minimizar o impacto de variações aleatórias e aumentar a confiabilidade dos dados coletados. Os resultados de cada medição individual estão organizados na Tabela 1, que serão utilizados para calcular os valores processados da Tabela 2.

² Código completo disponível em: https://nop.dainf.ct.utfpr.edu.br/nop-implementations/frameworks/nop-framework-micropython-1_0/nop-framework-micropython-1_0-applications/sensors

Tabela 1 – Tempos de latência da aplicação de sensores.

Tempo de Execução (milissegundos)										
Versão	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
PON Local	3,763	3,747	3,679	3,757	3,541	3,675	3,763	3,635	3,808	3,333
PON Distribuído	17,926	21,846	14,706	20,527	14,349	21,599	17,716	15,692	15,099	15,634
POO Local	1,570	1,535	1,814	1,517	1,559	1,800	1,571	1,750	1,582	1,814
POO Distribuído	7,581	7,649	7,731	7,693	7,678	7,713	7,699	7,701	7,195	7,749

Fonte: O autor.

Ao analisar preliminarmente os dados, nota-se que as versões que utilizam o Framework PON MicroPython 1.0 apresentam uma latência 122% maior em comparação às implementações orientadas a objetos. Esse comportamento era esperado, já que o *framework* introduz uma carga adicional devido ao processamento requerido pela cadeia de notificações.

Para uma análise mais profunda, foram calculados a média, o desvio padrão e o percentil 99 dos tempos de latência para cada implementação. Esses valores estão apresentados na Tabela 2. Observa-se que a média dos tempos de latência está próxima ao percentil 99 (P99), enquanto o desvio padrão apresenta valores abaixo de 5%. Isso sugere que os tempos de latência medidos são consistentes, com poucas variações entre as medições. Adicionalmente, verificou-se que o desvio padrão na versão do PON é maior em comparação à versão POO. Apesar de uma maior dispersão, os resultados permanecem dentro de padrões adequados de estabilidade e previsibilidade para ambas as versões.

Tabela 2 – Tempos de latência da aplicação de sensores processados.

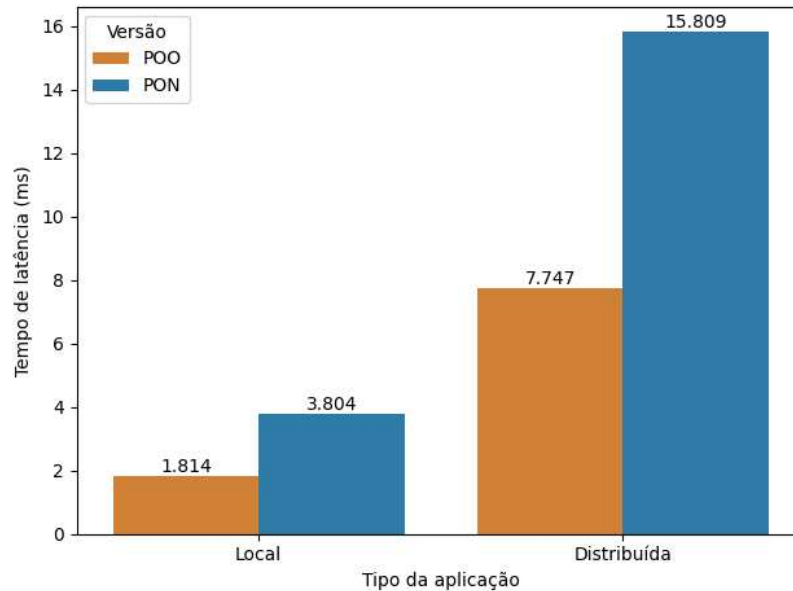
Versão	P99 (ms)	Média (ms)	Desvio Padrão (ms)
PON Local	3,804	3,670	0,135
PON Distribuído	15,809	14,797	0,482
POO Local	1,814	1,651	0,119
POO Distribuído	7,747	7,639	0,154

Fonte: O autor.

Por fim, para facilitar a visualização das diferenças de latência entre as implementações, foi utilizado o percentil 99 (P99) dos tempos medidos para gerar o gráfico apresentado na Figura 15. Observa-se claramente que as implementações utilizando o Framework PON MicroPython apresentam uma latência maior, sendo

aproximadamente 110% superior na versão local e 104% maior na versão distribuída, quando comparadas às versões orientadas a objetos.

Figura 15 – Tempo de latência (p99) da aplicação de sensores.



Fonte: O autor.

Além das medições de latência, também foi avaliado o número de linhas de código necessárias para cada implementação, como apresentado na Tabela 3. Note-se que, para ambas as versões, foi utilizado um formatador de código e as linhas em branco foram desconsideradas na contagem, eliminando quaisquer influências decorrentes de diferenças estilísticas ou estruturais no código. Na análise desses dados, observa-se que não há uma diferença significativa entre as versões locais. Contudo, ao analisar as versões distribuídas, verifica-se uma diferença significativa: a implementação orientada a objetos possui quase três vezes mais linhas de código em comparação à versão com o Framework PON MicroPython 1.0.

Tabela 3 – Linhas de Código por versão da aplicação *Sensors*.

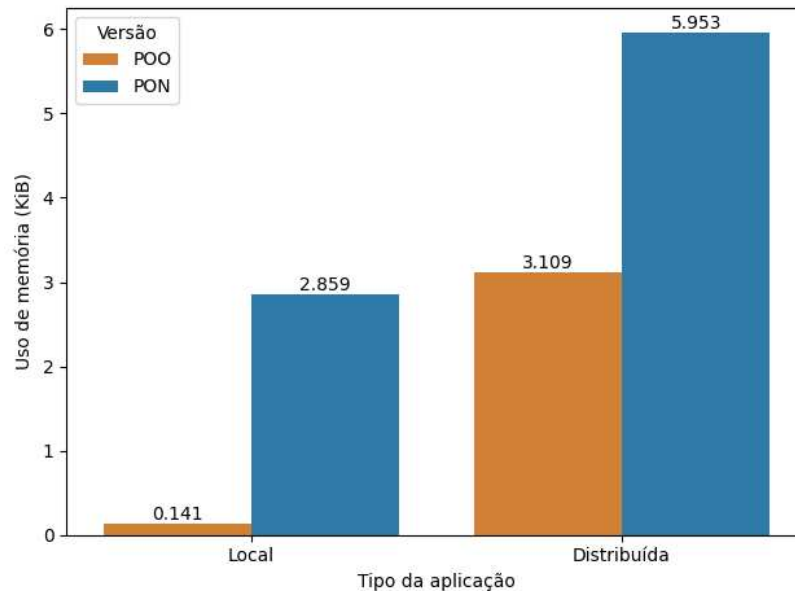
Versão	Linhas
PON Local	34
PON Distribuído	41
POO Local	36
POO Distribuído	115

Fonte: O autor.

Adicionalmente, foi realizado um levantamento do uso de memória RAM para cada versão da aplicação, conforme ilustrado na Figura 16. Observa-se que o *Framework* PON MicroPython 1.0 utiliza significativamente mais memória em

comparação à versão POO. Essa diferença se deve às diversas estruturas adicionais exigidas para os elementos que compõem a cadeia de notificações no Framework PON MicroPython 1.0.

Figura 16 – Uso de memória RAM da aplicação de sensores.



Fonte: O autor.

A análise dos resultados obtidos evidência de forma clara tanto os impactos positivos quanto os negativos do Framework PON MicroPython 1.0. A análise da extensão do código revelou uma vantagem significativa na implementação distribuída, com uma redução na quantidade de linhas de código. Essa diminuição reflete a capacidade do *framework* de encapsular funcionalidades complexas de comunicação entre microcontroladores, simplificando o processo de desenvolvimento.

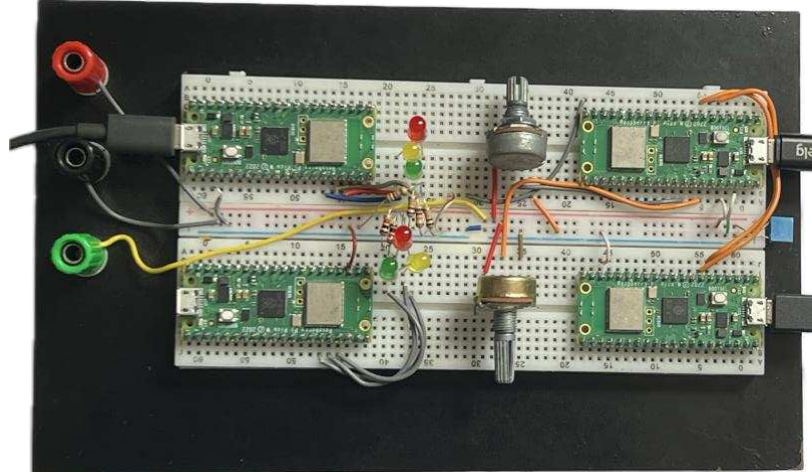
No entanto, a abstração oferecida pelo Framework PON MicroPython 1.0 apresenta um custo adicional em termos de latência e uso de memória. Esses fatores podem representar desafios importantes em sistemas com restrições severas de recursos, como microcontroladores de baixa capacidade. Portanto, embora o *framework* traga melhorias na produtividade e manutenção do código, deve ser considerado o seu maior uso de memória RAM, principalmente em cenários onde o consumo de recursos seja crítico.

4.2 Aplicação de monitor de qualidade do ar

A implementação da aplicação de monitor de qualidade de ar seguiu os diagramas definidos na fase de projeto, resultando em duas versões: uma utilizando o Framework PON MicroPython 1.0 desenvolvido e outra baseada exclusivamente no

paradigma orientado a objetos. Para ambas as versões, foram empregados quatro microcontroladores Raspberry Pi Pico W independentes, com os sensores simulados por potenciômetros e os LEDs RGB representados por três LEDs individuais, como ilustrado na Figura 17. Todo o código desta aplicação encontra-se no seu próprio repositório³.

Figura 17 – Conexões entre microcontroladores da aplicação *AirQualityMonitor*.



Fonte: O autor.

A implementação da versão que utiliza o Framework PON MicroPython 1.0 aproveitou integralmente seus recursos. Os potenciômetros foram configurados com o *ADCAttribute*, utilizando opções de ajuste de escala para adequar os valores lidos. Os LEDs, por sua vez, foram controlados por meio do *GPIOAttribute* configurado no modo de saída. A comunicação entre os microcontroladores foi realizada utilizando os drivers de comunicação *UARTDriver* e *UDPDriver*, permitindo a distribuição eficiente das notificações entre os elementos do sistema.

Na versão baseada no paradigma orientado a objetos, foi investido o mesmo tempo de desenvolvimento, seguindo boas práticas de programação. Cada *FBE* foi implementado como uma classe individual, sem o suporte do *framework*. A leitura dos potenciômetros foi realizada pelo método de *polling* dos ADCs, enquanto os LEDs foram controlados utilizando as bibliotecas padrão do MicroPython. Para implementar a comunicação distribuída, foram desenvolvidos *drivers* específicos, similares aos do *framework*, mas baseados em funções de *callback* para gerenciar mudanças nos estados.

³ https://nop.dainf.ct.utfpr.edu.br/nop-implementations/frameworks/nop-framework-micropython-1_0/nop-framework-micropython-1_0-applications/aqm

Para medir a latência no sistema, o sensor foi substituído por um botão, a fim de simular uma mudança brusca no sensor de maneira mais consistente e controlada, para manter controle nos testes. A medição foi realizada utilizando um microcontrolador Raspberry Pi Pico W externo dedicado exclusivamente a esse propósito. Esse microcontrolador foi conectado diretamente ao botão e ao LED que seria ativado em resposta à mudança de estado do sensor. Ele foi configurado para registrar o tempo transcorrido entre a detecção do acionamento do botão e a ativação do LED. Para garantir a precisão da medição, foram utilizadas interrupções tanto para capturar o evento de acionamento do botão quanto para detectar a ativação do LED, minimizando possíveis atrasos associados ao processamento.

Foram realizadas dez medições para cada implementação, com o objetivo de minimizar o impacto de variações aleatórias e aumentar a confiabilidade dos dados coletados. Os resultados de cada medição individual estão organizados na Tabela 4, que serão utilizados para calcular os valores processados da Tabela 5.

Tabela 4 – Tempos de latência da aplicação AQM.

Tempo de Execução (milissegundos)										
Versão	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
PON	130,4	37,50	44,4	115,9	71,10	130,5	108,5	138,8	62,40	33,30
POO	72,00	115,4	44,1	91,90	22,40	73,20	80,00	100,6	25,40	59,80

Fonte: O autor.

Para facilitar a análise, foram calculados a média, o desvio padrão e o percentil 99 dos tempos de latência para cada implementação. Esses valores estão apresentados na Tabela 5. Observa-se que a média dos tempos de latência está bastante distante ao percentil 99 (P99), o que é mais esperado, já que a comunicação do *UDPDriver* utilizado nesta aplicação é realizada por Wi-Fi. Adicionalmente, foi observado um desvio padrão de aproximadamente 45% nas duas versões.

Tabela 5 – Tempos de latência da aplicação AQM processados.

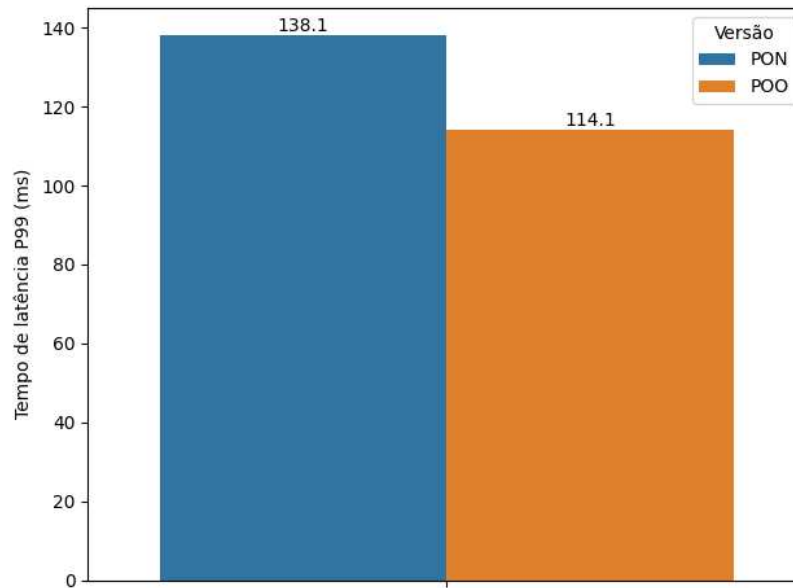
Versão	P99 (ms)	Média (ms)	Desvio Padrão (ms)
PON	138,1	87,27	37,70
POO	114,0	68,48	29,30

Fonte: O autor.

Pode-se observar que a versão com o *Framework* PON MicroPython 1.0 apresenta uma desvantagem de aproximadamente 18 milissegundos em relação à versão POO. Contudo, nessa aplicação específica, a diferença percentual é menor

que a vista na aplicação de sensores, sendo de 22% na média e 17% no percentil 99 (P99), como pode ser observado na Figura 18. Essa redução percentual pode ser atribuída ao fato de a aplicação representar um caso de uso mais realista, envolvendo o compartilhamento de diversos valores, como os provenientes de sensores e os estados relacionados à qualidade do ar.

Figura 18 – Tempo de latência da aplicação *AirQualityMonitor*.



Fonte: O autor.

A seguir foi realizada a análise comparativa das extensões de linhas de código das implementações, que revelou diferenças de entre 68% e 153% na extensão do código. Conforme indicado na Tabela 6, a versão sem o Framework PON MicroPython 1.0 apresentou um aumento significativo na quantidade de linhas, totalizando 251 linhas a mais do que a versão que utiliza o *framework*. Esse aumento representa 153%, evidenciando a eficiência proporcionada pelo *framework* na redução da complexidade e extensão do código. Mesmo desconsiderando as linhas de código atribuídas à categoria "Outros" (relacionadas a classes de apoio na comunicação), ainda se observa uma diferença de 92 linhas, o que equivale a um aumento de 68%.

Tabela 6 – Linhas por componente da aplicação *AirQualityMonitor*.

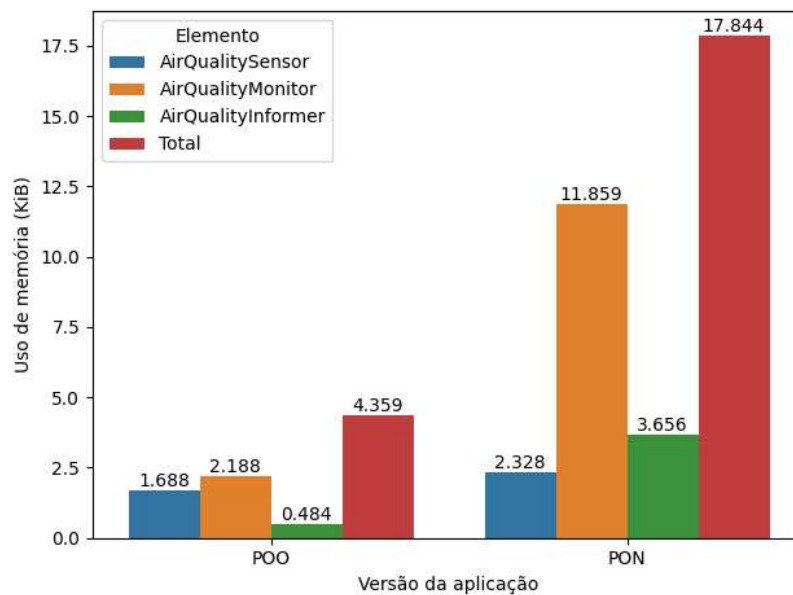
Componente	PON (Linhas)	POO (Linhas)	Diferença (Linhas (%))
<i>AirQualitySensor</i>	9	52	43 (478%)
<i>AirQualityMonitor</i>	51	88	37 (73%)
<i>AirQualityInformer</i>	42	45	3 (7%)
<i>Main</i>	33	42	9 (27%)
Subtotal	135	227	92 (68%)
Outros	0	114	114 (-)
Total	135	341	206 (153%)

Fonte: O autor.

No caso do *AirQualitySensor*, a diferença é atribuída à facilidade proporcionada pelo *ADCAttribute* no *framework*. Este encapsula de forma eficiente as configurações e funcionalidades necessárias para a leitura do ADC, reduzindo a quantidade de código necessária. Já no *AirQualityMonitor*, a redução se deve à capacidade do PON de eliminar redundâncias na definição de regras.

Adicionalmente, foi realizado um levantamento do uso de memória RAM para cada versão da aplicação, conforme ilustrado na Figura 19. Observa-se que o *Framework* PON MicroPython 1.0 consome mais memória em comparação à versão POO. Essa diferença é atribuída às estruturas adicionais necessárias para os elementos do *framework*, como as relacionadas ao gerenciamento de notificações e comunicação distribuída. Esse impacto é mais evidente no elemento *AirQualityMonitor*, que apresenta um número maior de regras e utiliza simultaneamente os dois *drivers* de comunicação disponíveis (*UARTDriver* e *UDPDriver*).

Figura 19 – Uso de memória RAM da aplicação de AQM.



Fonte: O autor.

A análise da aplicação AQM destaca os benefícios proporcionados pelo uso do *Framework* PON MicroPython 1.0 em termos de simplicidade, organização e redução do esforço de desenvolvimento. A estrutura modular e as abstrações do *framework* contribuem para um código mais claro e menos propenso a erros, facilitando a implementação de funcionalidades distribuídas e a comunicação entre os componentes. No entanto, apresenta uma desvantagem em relação à latência, e adicionalmente a aplicação evidencia o elevado consumo de memória associado ao uso do *framework*.

4.3 Considerações dos resultados

A análise dos resultados das duas aplicações, *AirQualityMonitor* e *Sensors*, evidencia os impactos do *Framework* PON MicroPython 1.0 tanto em termos de latência, uso de memória e de facilidade de uso. Em ambas as aplicações, o *framework* demonstrou ser eficaz na simplificação do desenvolvimento, abstraindo funções externas ao objetivo das aplicações em componentes reutilizáveis. Essa vantagem é especialmente visível nas implementações distribuídas, onde o *framework* eliminou grande parte da complexidade associada à configuração das interfaces de comunicação entre microcontroladores e atualização de estados dos componentes.

No entanto, o uso do *framework* apresentou um custo em termos de desempenho, com aumento na latência, principalmente observado na aplicação de sensores. Assim como o aumento do uso de memória, que foi principalmente

observado na aplicação AQM. Esse impacto é esperado, dado o acréscimo do mecanismo de notificações e da modularidade introduzida pelos *drivers* de comunicação. Apesar disso, as medições demonstraram consistência nos tempos de resposta, indicando que o *framework* mantém um comportamento previsível, essencial em sistemas embarcados.

Dessa forma, os resultados sugerem que o *Framework* PON MicroPython 1.0 é uma solução promissora para o desenvolvimento de aplicações distribuídas, na qual a redução do esforço de programação e a modularidade são mais importantes que o impacto na latência e onde a capacidade de memória não seja um problema. A adoção do *framework* permite criar sistemas distribuídos com maior facilidade, atendendo aos requisitos de projetos modernos e complexo, sempre que a latência e uso de memória seja compatível com os requisitos.

5 CONCLUSÃO

O desenvolvimento deste trabalho de conclusão de curso representou a aplicação prática de uma vasta gama de conhecimentos adquiridos ao longo do curso de Engenharia de Computação. A base fornecida pela engenharia de software possibilitou um planejamento sólido e estruturado, permitindo que o projeto e a implementação dos diferentes componentes e aplicações ocorressem sem problemas significativos. A integração de conceitos de eletrônica e sistemas embarcados foi essencial para compreender as particularidades e restrições desses sistemas, promovendo melhorias no processo de desenvolvimento e resultando em soluções mais eficientes. Além disso, as técnicas de análise de dados foram cruciais para a medição e avaliação rigorosa do desempenho do *framework*, garantindo a validação dos objetivos propostos.

O resultado deste trabalho é a implementação do Framework PON MicroPython 1.0, que preenche uma lacuna nos *frameworks* PON para rodar em sistemas microcontrolados. Esse novo *framework* possibilita, justamente, a análise das vantagens e desvantagens do PON neste contexto de microcontrolados, mas com a vantagem de utilizar uma linguagem de programação de alto nível. Além disso, o mecanismo de distribuição de notificações desenvolvido permite a avaliação de casos de uso distribuídos entre múltiplos microcontroladores. Por fim, destaca-se que, devido ao uso do MicroPython, as aplicações podem ser executadas diretamente em diferentes arquiteturas microcontroladas, desde que compatíveis com a linguagem.

5.1 Resultados

Durante todo o desenvolvimento deste trabalho foram considerados os objetivos específicos definidos na Seção 1.3.2 para garantir o cumprimento do objetivo geral definido na Seção 1.3.1. A seguir, são detalhadas as ações realizadas para atingir cada um desses objetivos.

O primeiro objetivo foi desenvolver o *Framework* PON MicroPython 1.0, incluindo todos os seus componentes principais (*Attribute*, *Premise*, *Condition*, *Rule*, *Action*, *Instigation*, *Method*). Esse *framework* foi projetado e implementado em sua totalidade como demonstrado nas aplicações de testes, além de integrar componentes específicos que facilitam a interação com periféricos dos microcontroladores.

Outro ponto central foi a criação de um mecanismo modular para comunicação de notificações entre os elementos *Attribute*, *Premise* e *Condition* do *framework*. Para isso, foi desenvolvida uma interface modular que permite a comunicação entre microcontroladores por meio de diferentes meios físicos e protocolos. Essa abordagem inclui a implementação de *drivers*, como o *UARTDriver* e o *UDPDriver*, que permitem sua adaptação a diferentes cenários de comunicação distribuída.

A aplicação prática do *framework* em um ambiente com pelo menos três microcontroladores foi demonstrada através do desenvolvimento da aplicação *Air Quality Monitor*. Essa aplicação utilizou quatro microcontroladores Raspberry Pi Pico W e dois *drivers* de comunicação distintos, ilustrando a capacidade do *framework* de integrar múltiplos dispositivos e compartilhar informações em vários meios de comunicação.

Por fim, a documentação do *Framework* PON MicroPython 1.0 foi desenvolvida com base na Seção 3.2 e está disponível no arquivo “README.md” do repositório oficial⁴ do *kernel* do *framework*. Essa documentação foi estruturada para ser acessível e prática, fornecendo informações completas sobre a instalação, configuração, uso e exemplos práticos do *framework*. Essa etapa é fundamental para garantir que outros desenvolvedores possam adotar o *framework* de maneira rápida e eficiente, contribuindo para a disseminação e uso da tecnologia desenvolvida.

Essas ações mostram que todos os objetivos específicos foram alcançados de forma satisfatória, contribuindo para o avanço no uso do PON em sistemas embarcados.

5.2 Trabalhos futuros

Este trabalho abre portas para diversas possibilidades de desenvolvimento e aprimoramento futuro, evidenciando o potencial e a flexibilidade do *framework* desenvolvido. Entre as direções mais relevantes estão a implementação de novos *drivers* de comunicação, como suporte ao *Bluetooth Low Energy* (BLE), expandindo as opções de conectividade para aplicações em diferentes casos de uso. Além disso, o desenvolvimento de mais aplicações utilizando o *framework* poderá não apenas

⁴ https://nop.dainf.ct.utfpr.edu.br/nop-implementations/frameworks/nop-framework-micropython-1_0/nop-framework-micropython-1_0-kernel

validar suas funcionalidades em diferentes cenários, mas também identificar novos pontos de melhoria e adaptações.

Outra possibilidade importante é a introdução de máscaras para os elementos notificados pelos *drivers* de comunicação, permitindo as notificações sejam enviadas apenas para os elementos relevantes. Essas máscaras funcionariam de maneira similar às máscaras de sub-rede, já que identificam os *ids* dos dispositivos associados a cada *driver*. Isso pode otimizar ainda mais o desempenho do *framework* em sistemas distribuídos.

Por fim, uma evolução significativa seria ajustar o código base para compatibilidade total com o Python, ampliando o uso do *framework* para aplicações em computadores com sistemas operacionais convencionais, o que abriria oportunidades para integração entre microcontroladores e sistemas mais robustos.

Em suma, esses trabalhos futuros aqui aventados mostrariam o potencial do *framework* para continuar evoluindo e contribuindo com o desenvolvimento de sistemas embarcados distribuídos.

5.3 Considerações finais

O *Framework* PON MicroPython 1.0 desenvolvido demonstra uma aplicação prática e inovadora do Paradigma Orientado a Notificações (PON), tornando-o acessível a uma ampla variedade de microcontroladores graças ao uso de MicroPython. A integração direta com periféricos relevantes e a modularidade do mecanismo de distribuição reforçam sua flexibilidade e adaptabilidade, permitindo futuras expansões com interfaces de comunicação adicionais sem requerer alterações no código-base. Dessa forma, este trabalho não apenas cumpre seus objetivos iniciais, mas também contribui para facilitar o desenvolvimento de sistemas embarcados distribuídos, promovendo escalabilidade, modularidade e eficiência no uso do PON.

REFERÊNCIAS

- BABU, Abu Ahammed. **Notification Oriented Paradigm as a Green Technology**. Master thesis, Green Networking and Cloud Computing (GENIAL), 2022.
- BANASZEWSKI, R. F. **Paradigma orientado a notificações: Avanços e comparações**. Dissertação (Mestrado) – CPGEI, UTFPR, 2009.
- BANERJEE, P.; CHANDY, J. A.; GUPTA, M.; et al. **The Paradigm Compiler for Distributed Memory Multicomputers**. IEEE Computer, v. 28, n. 10, p. 37-47, 1995.
- BARRETO, W. R. M.; VENDRAMIN, A. C. B. K.; SIMÃO, J. M. **Notification oriented paradigm for distributed systems**. In: Computer on the Beach 2018. [S.l.: s.n.], 2018.
- BELMONTE, D.; SIMÃO, J. M.; STADZISZ, P. C. **Proposta de um método para distribuição de carga de trabalho usando o paradigma orientado a notificações (PON)**. Revista SODEBRAS, v. 8, n. 84, 2012.
- BROOKSHEAR, J. G. **Computer Science: An Overview**. 9. ed. [S.l.]: Addison-Wesley, 2006.
- DAMIEN, George et al. **MicroPython**. 2023. Disponível em: <https://github.com/micropython/micropython>. Acesso em: 24 jan. 2025.
- FIGUEIREDO, Luis Henrique Pelegrin. **Framework PON C++ 4.0 IoT: Paradigma Orientado a Notificações para Ambiente de Internet das Coisas**. Dissertação (Mestrado em Computação Aplicada) – Universidade Tecnológica Federal do Paraná, Curitiba, 2022.
- GABBRIELLI, Maurizio; MARTINI, Simone. **Programming Languages: Principles and Paradigms**. Series: Undergraduate Topics in Computer Science. 1. ed. [S.l.]: Springer London, 2010. ISBN 978-1-84882-913-8.
- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. [S.l.: s.n.], 1995.
- HENZEN, A. F. **Portabilidade do framework PON de C++ standard para C# e Java**. Relatório técnico PPGCA/UTFPR, 2015.
- LINHARES, Robson Ribeiro. **Contribuição para o desenvolvimento de uma arquitetura de computação própria ao paradigma orientado a notificações**. Tese (Doutorado) – CPGEI, UTFPR, Curitiba, 2015.
- MAMANN, Lucas; SIMÃO, Jean; DELGADO, Myriam; PIGATTO, Daniel. **Paradigma orientado a notificações aplicado à programação de microcontroladores**. In: Anais Estendidos do XI Simpósio Brasileiro de Engenharia de Sistemas Computacionais. Porto Alegre, RS, Brasil: SBC, 2021. p. 134–139. ISSN 2763-9002.
- MARTINI, Guilherme H. K.; SIMÃO, J.; LINHARES, R. **A new method for dynamic balancing of workload and scalability in multicore systems**. Trabalho realizado na disciplina Tópicos Avançados em Sistemas Embarcados (CAES102 – PPGCA), 2018.

NEGRINI, Fabio. **Tecnologia NOPL Erlang-Elixir** – Paradigma orientado a notificações via uma abordagem orientada a microatores assíncronos. Dissertação (Mestrado) – Programa de Pós-Graduação em Engenharia Elétrica e Informática Industrial, Universidade Tecnológica Federal do Paraná, Curitiba, 2019.

NEVES, Felipe dos Santos; SIMÃO, Jean Marcelo; LINHARES, Robson Ribeiro. ***Application of generic programming for the development of a C++ framework for the Notification Oriented Paradigm***. In: Anais da XI International Conference on Information Society and Technology (ICIST 2021), 2021.

OSHIRO, Larissa Keiko. **Contribuição para o Paradigma Orientado a Notificações**: Evolução da Tecnologia LingPON 2.0 via Aprimoramento da Linguagem e Compilador para Código Notificante Modular em C++. Dissertação (Mestrado) – Programa de Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI), Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, 2021.

RASPBERRY PI (TRADING) LTD. RP2040 ***Datasheet: A microcontroller by Raspberry***. Raspberry, 2020.

RAYMOND, E. S. ***The Art of UNIX Programming***. Addison-Wesley, 2003.

RENESAS. ***MCU Programming Peripherals 01: GPIO***. Disponível em: <https://www.renesas.com/en/support/engineer-school/mcu-programming-peripherals-01-gpio>. Acesso em: 23 fev. 2025.

RONSZCKA, A. F. **Contribuição para a concepção de aplicações no paradigma orientado a notificações (PON) sob o viés de padrões**. Dissertação (Mestrado em Engenharia Elétrica e Informática Industrial) – CPGEI, UTFPR, Curitiba, 2012.

RONSZCKA, A. F. **Método para a Criação de Linguagens de Programação e Compiladores para o Paradigma Orientado a Notificações em Plataformas Distintas**. Tese (Doutorado) – CPGEI, UTFPR, Curitiba, 2019.

RONSZCKA, A. F.; FERREIRA, C. A.; STADZISZ, P. C.; et al. ***Notification-Oriented Programming Language and Compiler***. In: SBESC – VII Brazilian Symposium on Computing Systems Engineering. Anais. [S.l.]: SBC, 2017.

SCHÜTZ, F. **NeuroPON**: uma abordagem para o desenvolvimento de redes neurais artificiais utilizando o paradigma orientado a notificações. Tese (Doutorado) – Doutorado em Engenharia Elétrica e Informática Industrial, Universidade Tecnológica Federal do Paraná, Curitiba, 2019.

SCOTT, M. L. ***Programming Language Pragmatics***. [S.l.]: Morgan Kaufmann Publishers Inc, 2000.

SIMÃO, J. M. **Proposta de uma arquitetura de controle para sistemas flexíveis de manufatura baseada em regras e agentes**. Dissertação (Mestrado em Engenharia Elétrica e Informática Industrial) – CPGEI, UTFPR, Curitiba, 2001.

SIMÃO, J. M.; STADZISZ, P. C. **Paradigma orientado a notificações (PON)** – uma técnica de composição e execução de software orientado a notificações. Pedido de patente: Privilégio de Inovação, Número do registro: PI08055181, data de depósito:

26 nov. 2008. INPI – Instituto Nacional da Propriedade Industrial, Universidade Tecnológica Federal do Paraná, 2008.

SIMÃO, J. M.; STADZISZ, P. C.; BANASZEWSKI, R. F. ***Notification Oriented Paradigm (NOP) and Imperative Paradigm***: A Comparative Study. *Journal of Software Engineering and Applications*, v. 5, n. 6, p. 402-416, 2009.

TEXAS INSTRUMENTS. ***A Basic Guide to ADCs***. Disponível em: <https://www.ti.com/lit/an/slaa510/slaa510.pdf>. Acesso em: 23 fev. 2025.

TEXAS INSTRUMENTS. ***Motor interfaces and PWM frequencies***. Disponível em: <https://www.ti.com/video/6272403276001#transcript-tab>. Acesso em: 23 fev. 2025.

TEXAS INSTRUMENTS. ***Introduction to the Universal Asynchronous Receiver/Transmitter (UART)***. Disponível em: <https://www.ti.com/lit/ml/swrp216/swrp216.pdf>. Acesso em: 23 fev. 2025.

TEXAS INSTRUMENTS. ***I2C Communication Protocol***. Disponível em: <https://www.ti.com/lit/an/sbaa565/sbaa565.pdf>. Acesso em: 23 fev. 2025.

VALENÇA, Glauber Zárte. ***Contribuição para materialização do paradigma orientado a notificações (PON) via framework e wizard***. Dissertação (Mestrado em Computação Aplicada – PPGCA) – Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, 2012.

WATT, D. ***Programming Language Design Concepts***. [S.l.]: J. Willey & Sons, 2004.